

DATE : 04.08.2025

DT/NT : DT

LESSON : DEEP LEARNING

SUBJECT: COMPUTER VISION
IMAGE PROCESSING LIBRARIES
(PILLOW, SCIKIT-IMAGE, OPENCV)
&YOLO

BATCH : B287

**DATA
SCIENCE**



TECHPRO
EDUCATION



techproeducation.com



+1 (585) 304 29 59



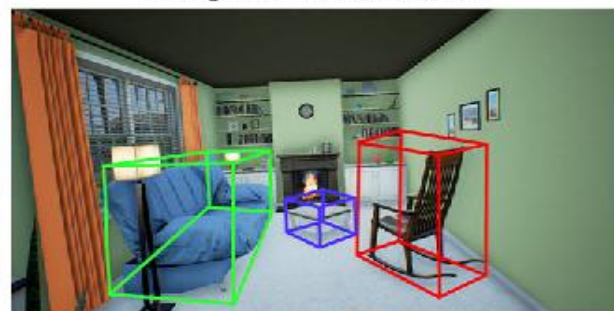
Object Tracking



Pose Estimation



Object Detection



Action Recognition



Autonomous Navigation



3D Reconstruction



Crowd Understanding



Urban Scene Understanding



Indoor Scene Understanding



Multi-agent Collaboration



Human Training



Aerial Surveying

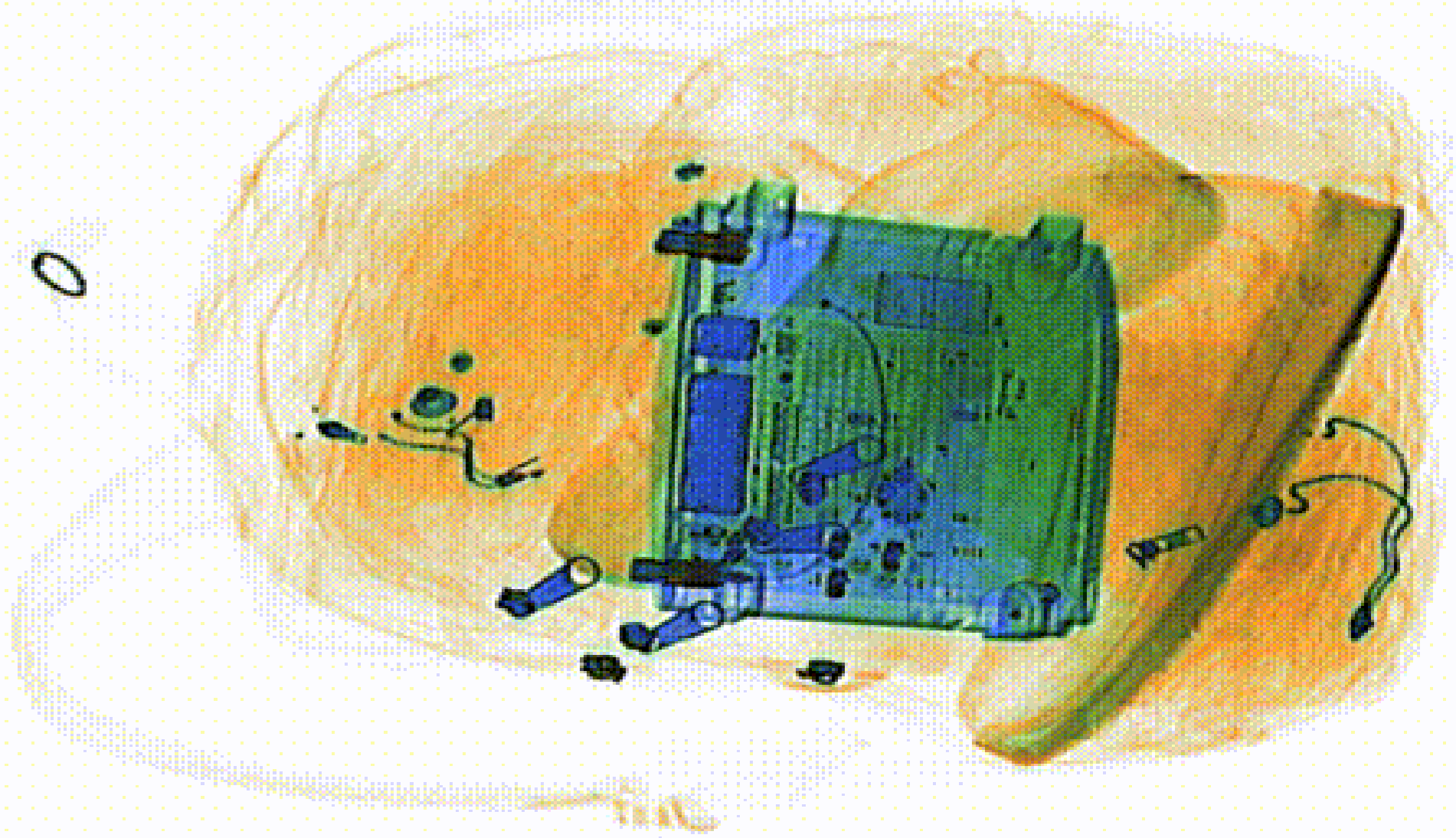


Red Image
Green Image Label

Orange Depth/Multi-View
Yellow User Input

Purple Video
Brown Physics

Pink Segmentation/Bounding Box
Gray Camera Localization



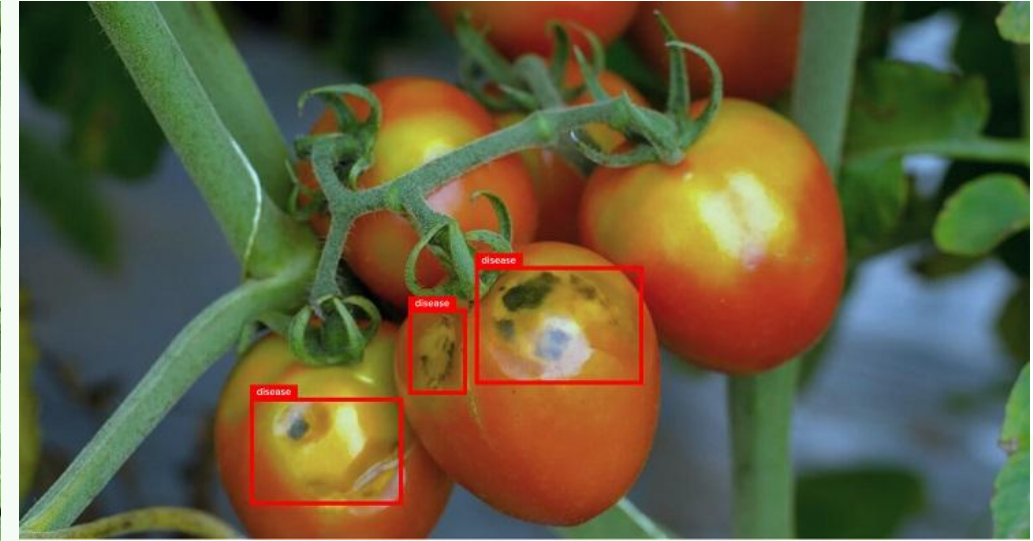
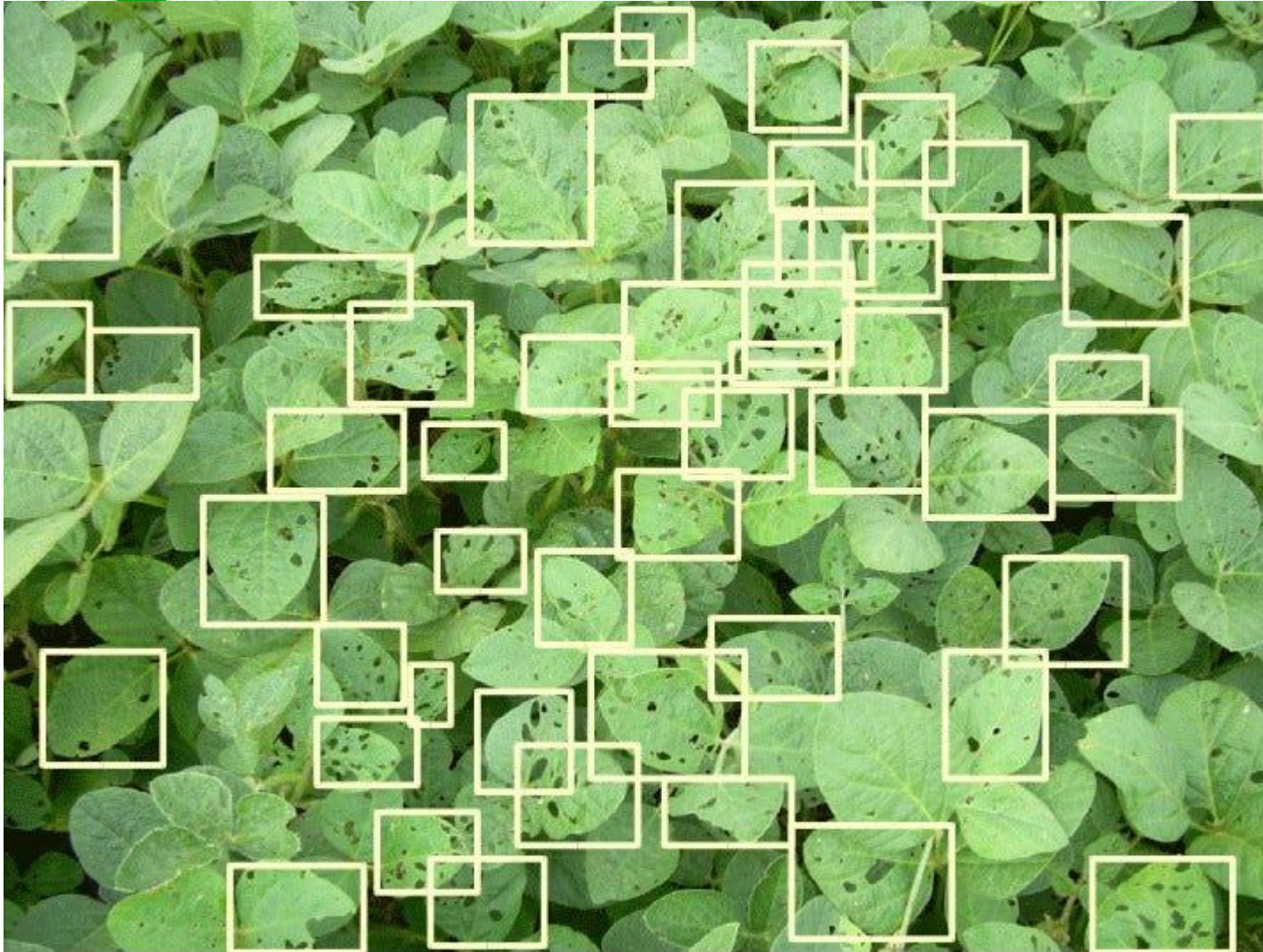


SELF-DRIVING CARS





CNN and Computer Vision





Greengrocer / Cofee Delivery Robots at Cambridge



Robots serving your Starbucks cofee

<https://www.youtube.com/watch?v=WBHhjRSZPtU>

The background is a complex digital composition. On the left, a large, stylized eye or camera lens is depicted with concentric circles and a central iris that shows a colorful, abstract pattern. Radiating from this center are numerous thin, glowing lines. To the right, a network of blue circuit traces and nodes is visible, with several points of light. The overall color palette is dominated by deep blues and teals, with some lighter, glowing elements.

Computer vision



Exploring Visual Processing Tasks: From CNNs to Computer Vision



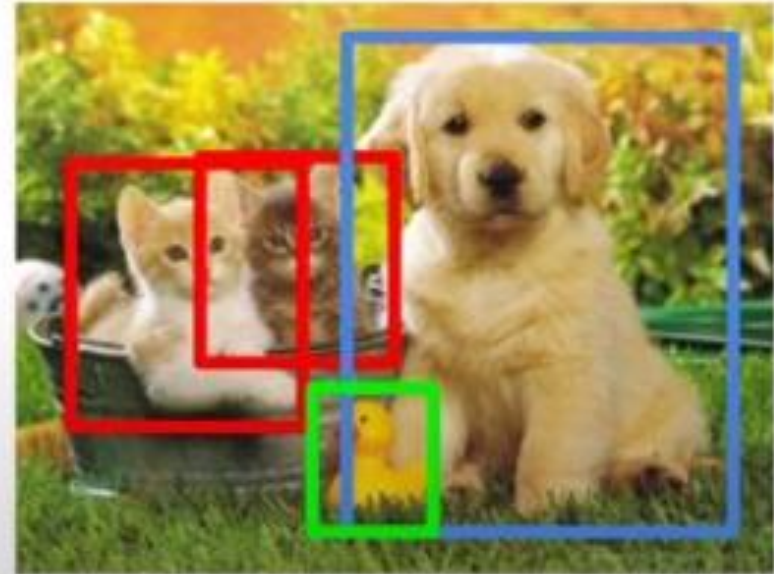
CNN and Computer Vision

Classification



CAT

Object Detection



CAT, DOG, DUCK



CNN and Computer Vision

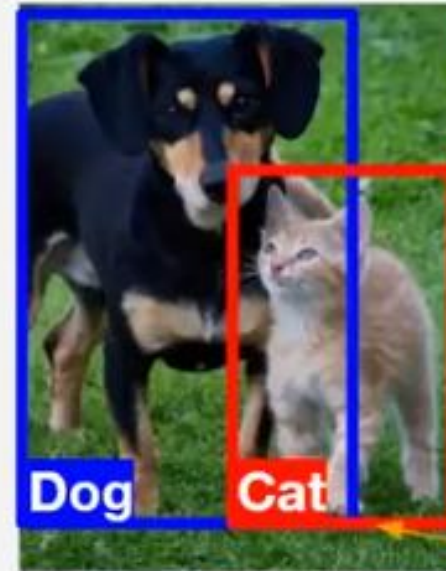
Classification

- One object and label per image



Object Detection

- Multiple objects per image
- Determine objects' location



Bounding Box

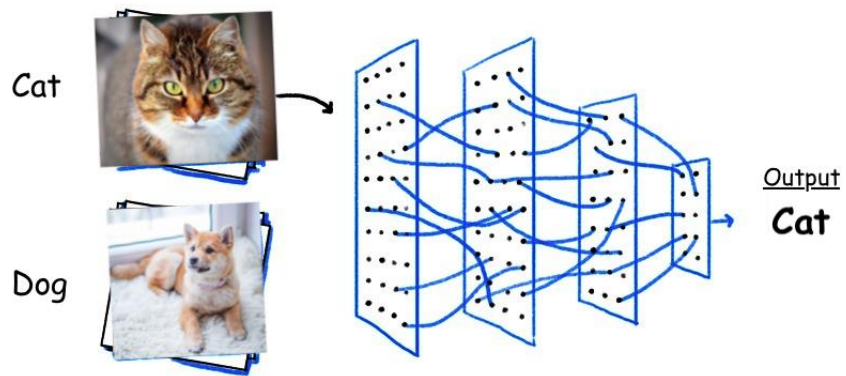
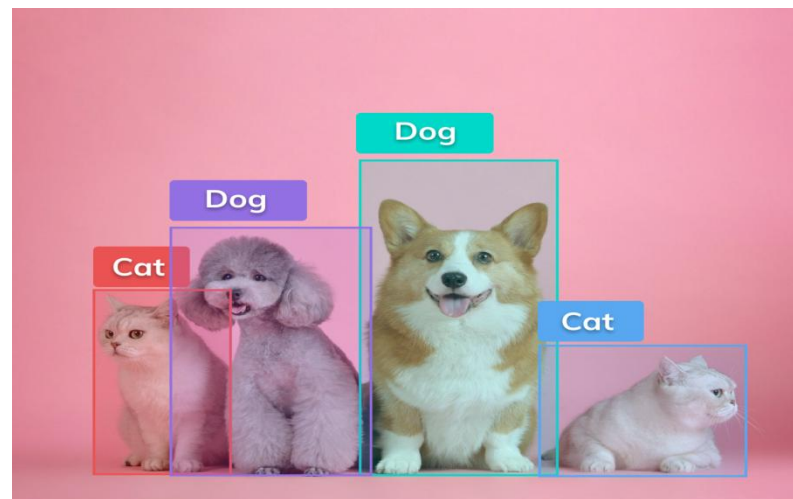
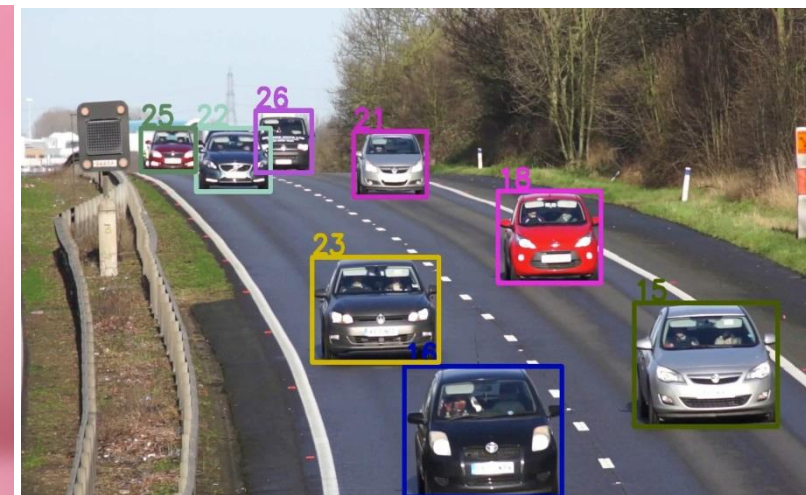


Image Classification



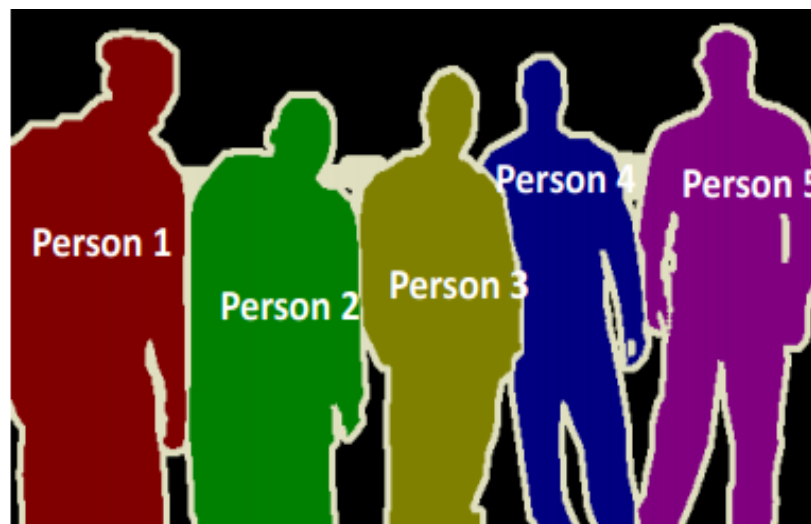
Object Detection



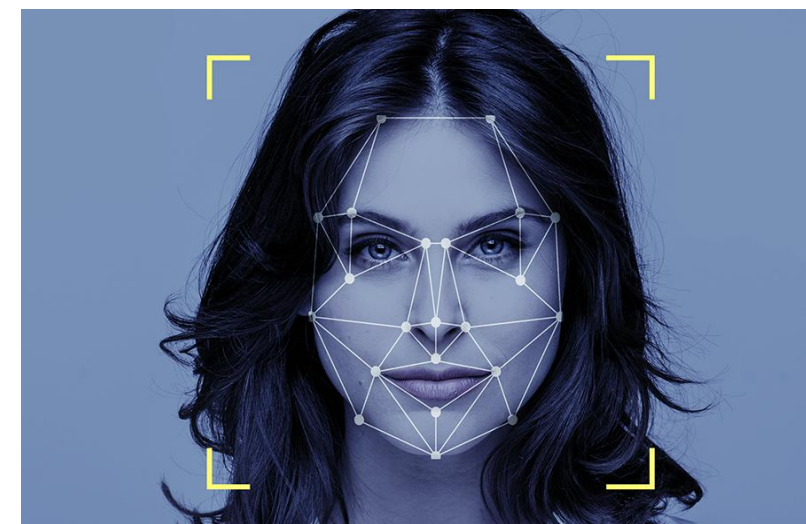
Object Tracking



Semantic Segmentation



Instance Segmentation



Face Detection and Recognition



CNN and Computer Vision

Semantic Segmentation



GRASS, CAT,
TREE, SKY

No objects, just pixels

Classification + Localization



CAT

Single Object

Object Detection



DOG, DOG, CAT

Multiple Object

Instance Segmentation



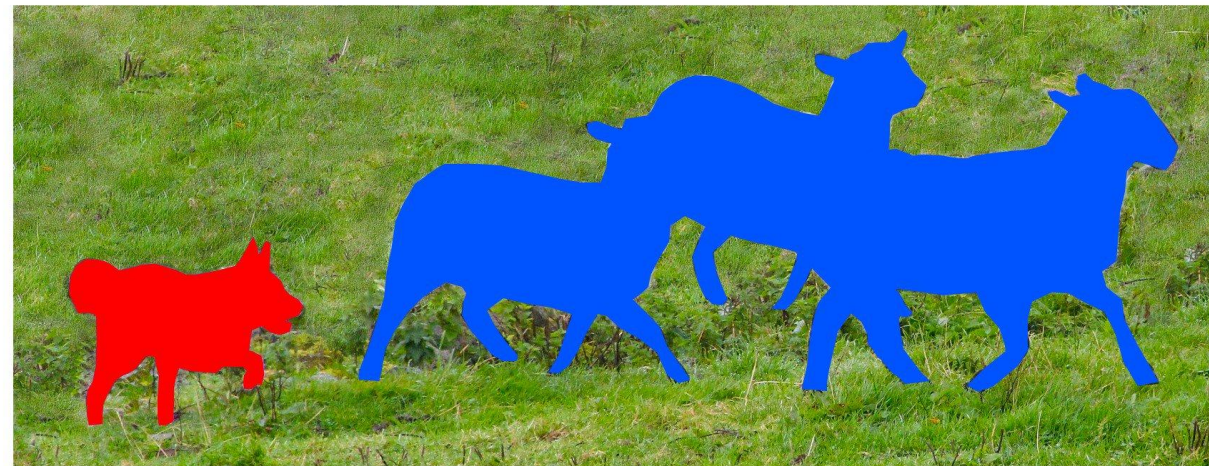
DOG, DOG, CAT



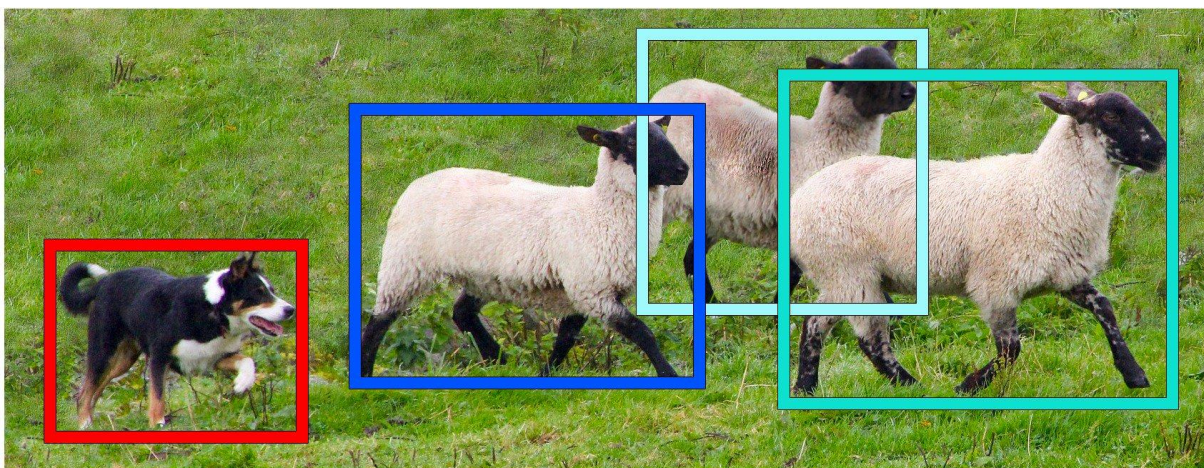
CNN and Computer Vision



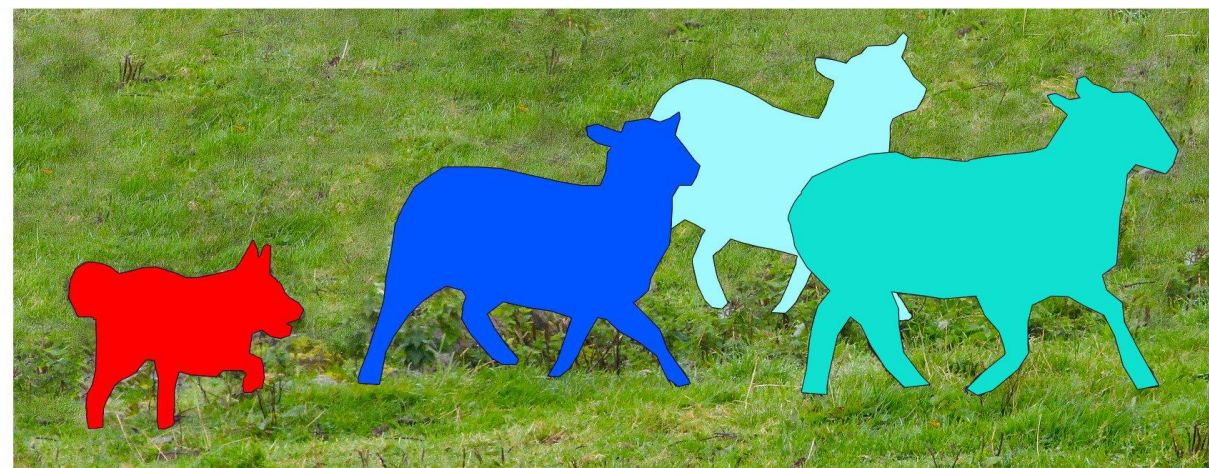
Image Recognition



Semantic Segmentation



Object Detection



Instance Segmentation



CNN and Computer Vision

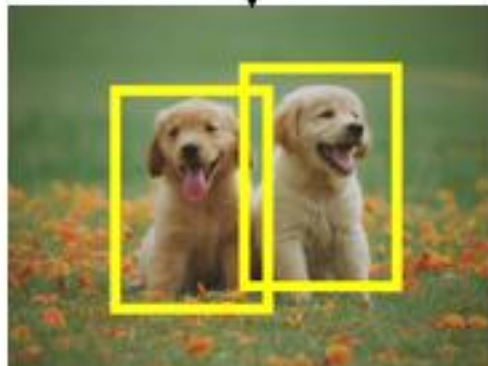
Input Image



Image Classification



Object Detection

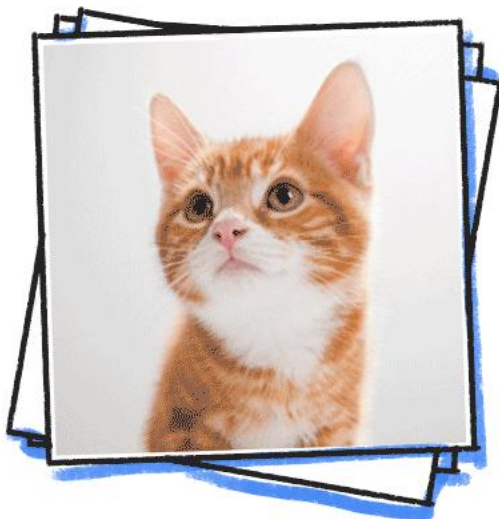


Instance Segmentation



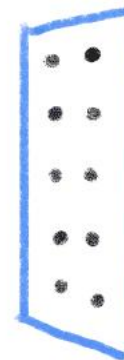
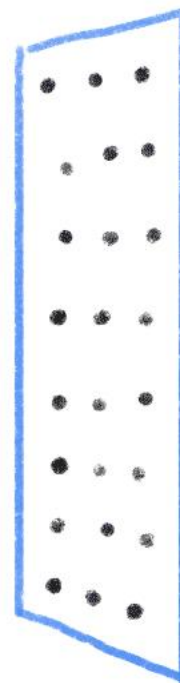
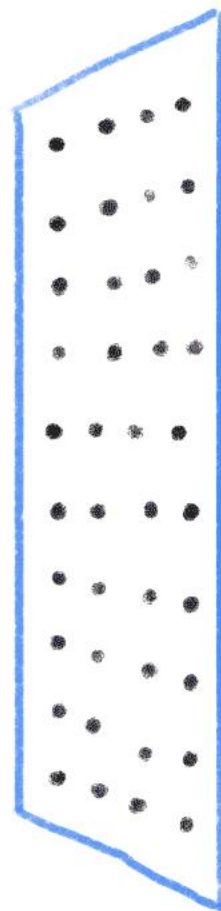
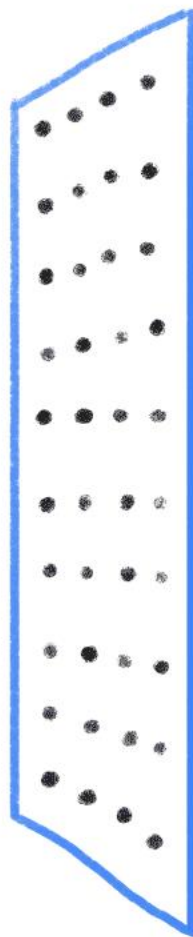


CAT



(Labeled
PHOTOS)

DOG

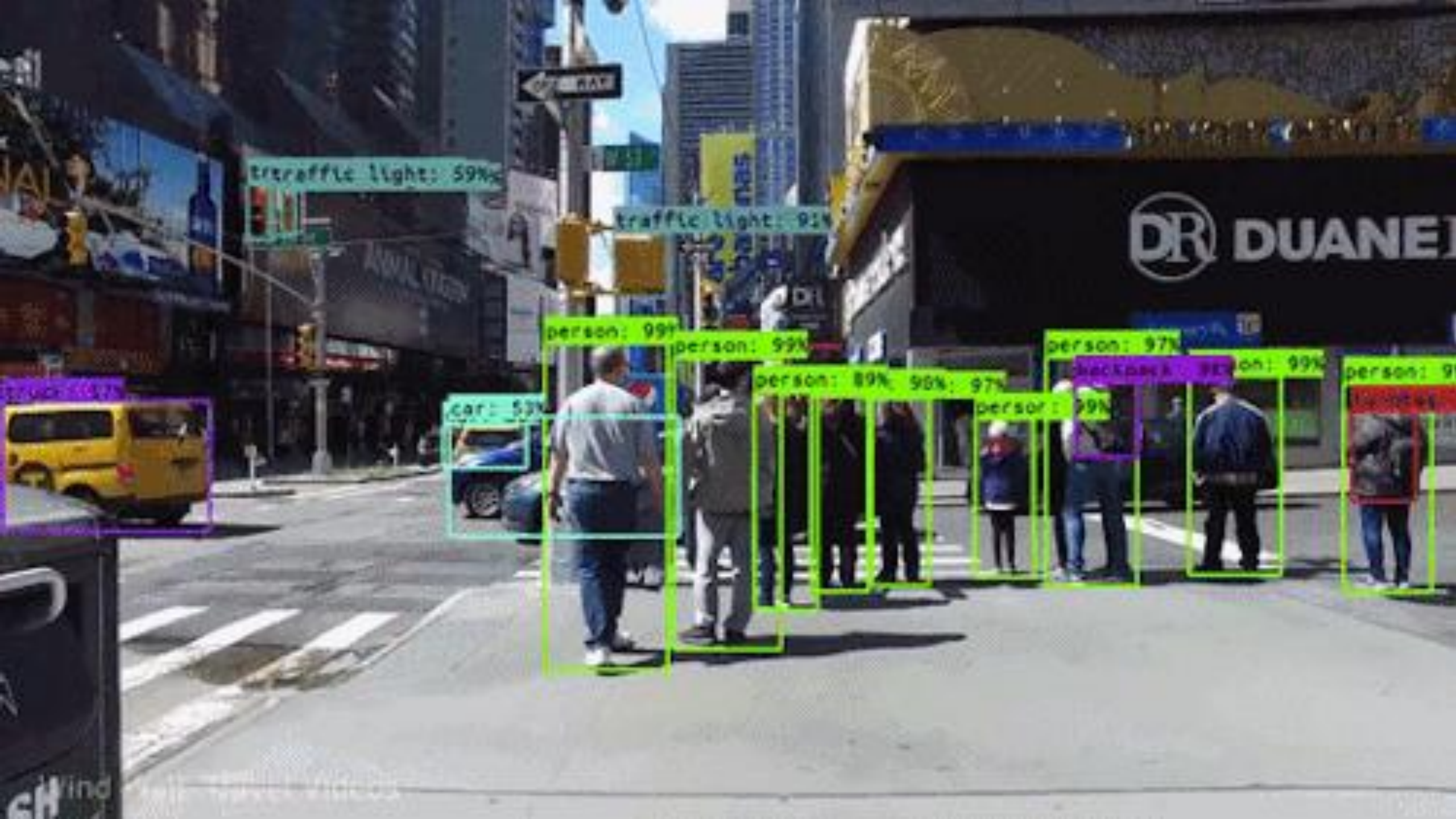


OUTPUT



OBJECT DETECTION





traffic light: 59%

traffic light: 91%

person: 99%

person: 99%

person: 97%

backpack: 98%

person: 99%

person: 99%

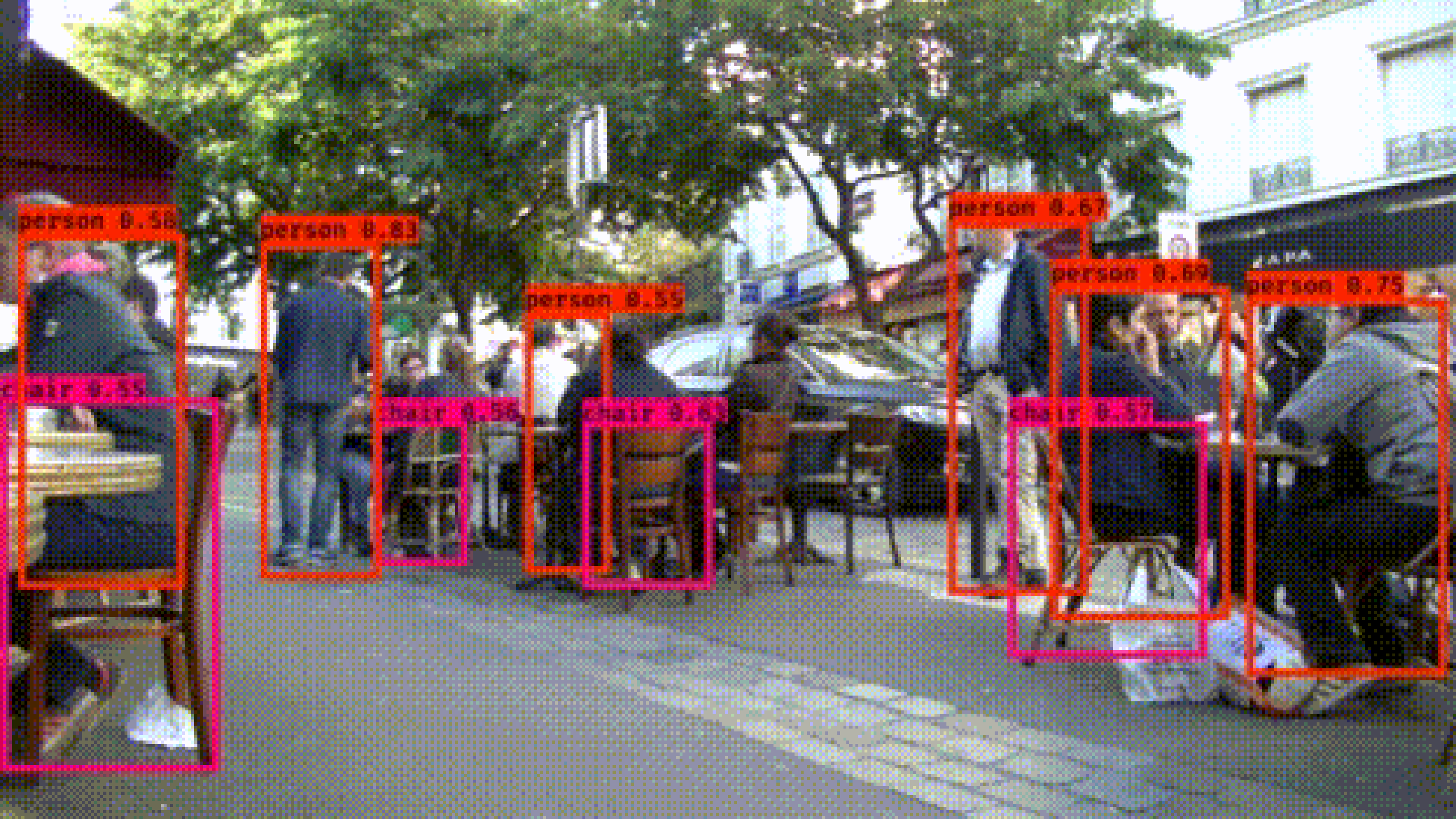
car: 53%

person: 89%, 98%, 97%

person: 99%

person: 99%





person 0.58

person 0.83

person 0.55

person 0.67

person 0.69

person 0.75

chair 0.55

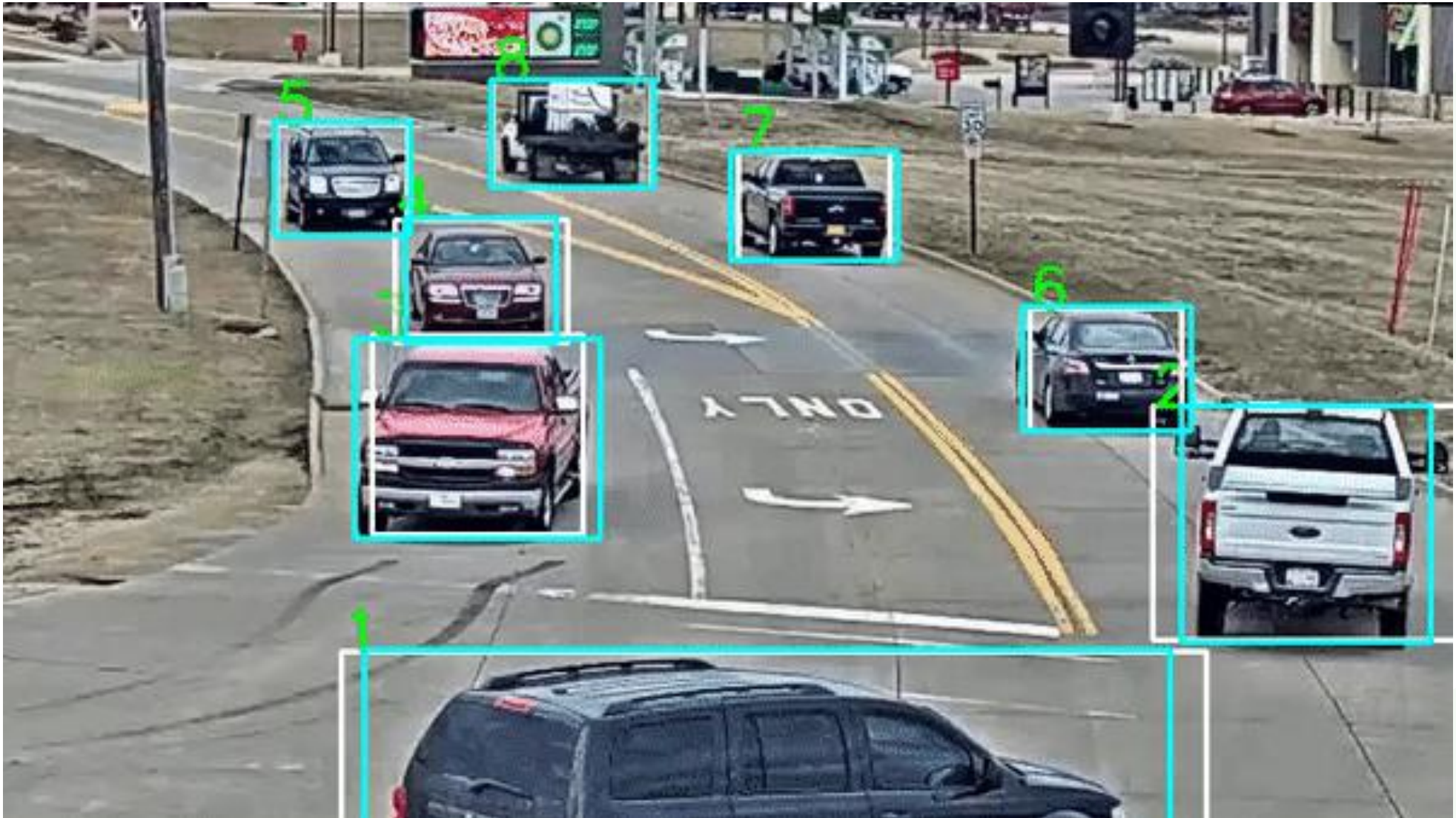
chair 0.56

chair 0.63

chair 0.57



OBJECT TRACKING





SEMANTIC SEGMENTATION



road
sidewalk
building
wall
fence
pole
traffic light
traffic sign
vegetation
terrain
sky
person
rider
car
truck
bus
train
motorcycle
bicycle



FACE DETECTION AND RECOGNITION





FACE DETECTION AND RECOGNITION

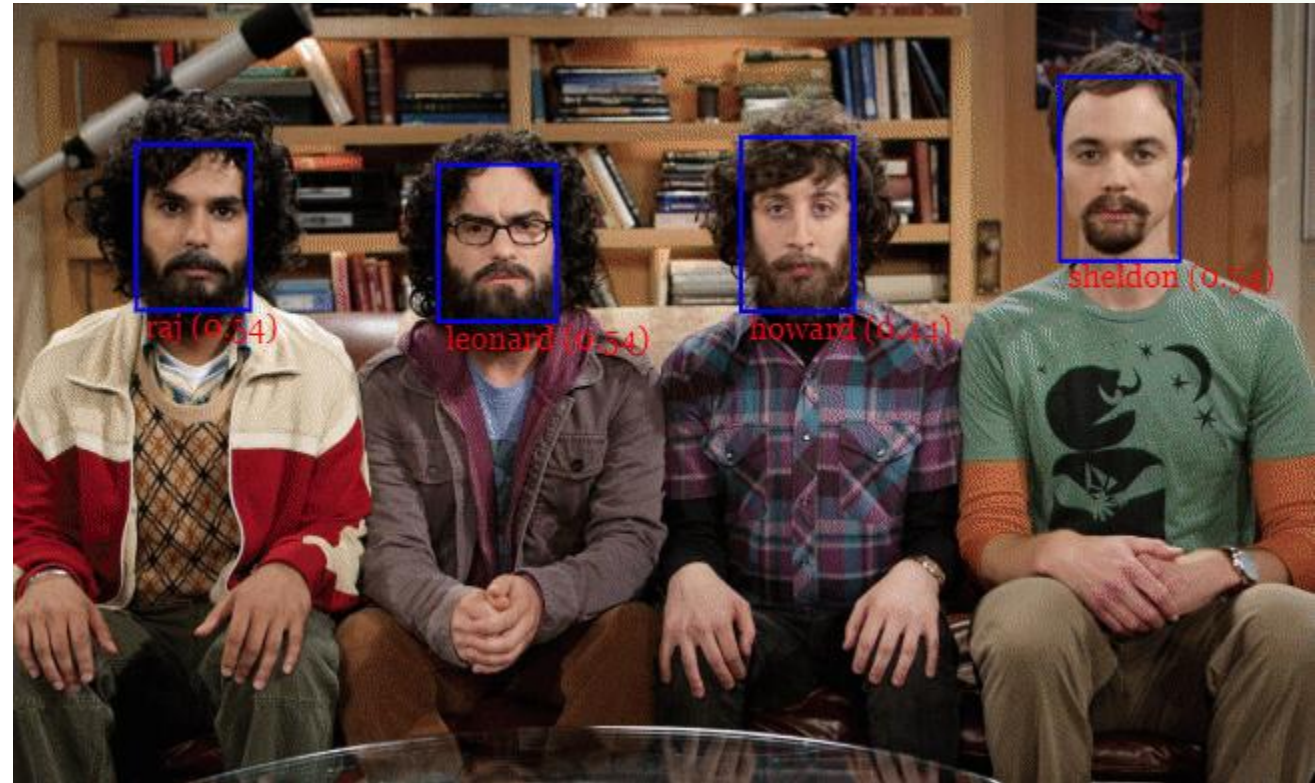
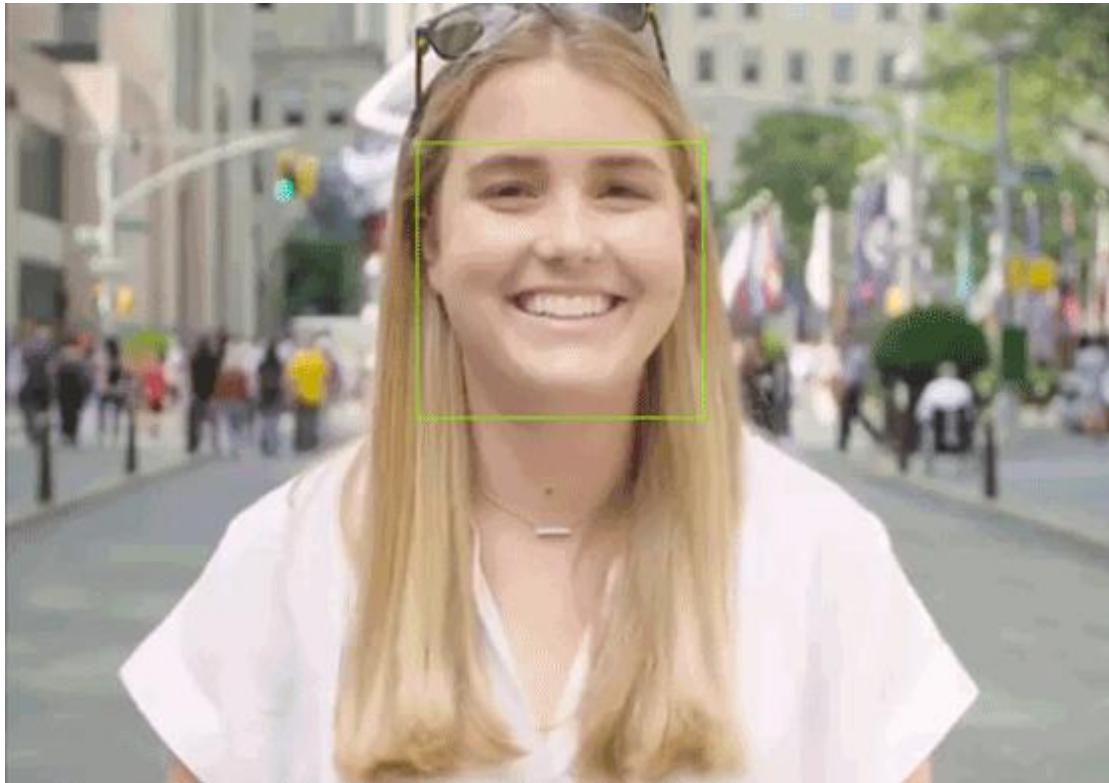




IMAGE PROCESSING

Image processing involves techniques and methods used by computers to understand, interpret, manipulate, analyze, and utilize image data.



Image processing includes many operations, such as digitising analogue images, determining the values of digital images, analysing, feature extraction, recognition, display, compression, cropping, and resizing.

It is used in many fields, such as enhancement imaging, robotics, remote sensing, security, quality control, engineering, industry, and entertainment.

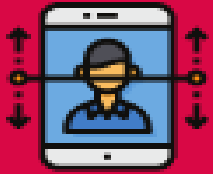


IMAGE PROCESSING

1

**Process an Image to
generate another image**

2

Input and output is image

3



Input [image]

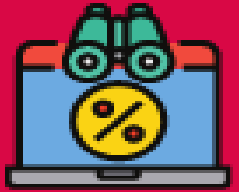


output [image]

4

**Smoothing, sharpening, compression,
contrast, watermarking brightness, edges,
region.**

COMPUTER VISION

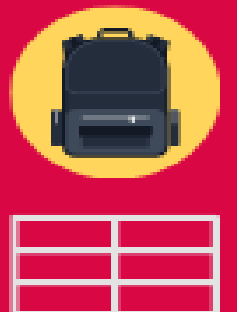


**Process an Image to produce
desired result in real time**

**Input as an image
Output as desired result**



Input [image]



**Position, Action, Identification,
Measure, Label, Projection**

Image Processing

Focuses on processing the image.
Both input and output are images.



SMOOTHING &
SHARPENING

CHANGE
CONTRAST &
BRIGHTNESS

HIGHLIGHT EDGES
& REGIONS

WATERMARKING

COMPRESSION

CALIBRATION

VS

Computer Vision

Focuses on making sense of what a
machine sees. The system inputs
an image and outputs task-specific
knowledge.



LABELLING

POSITION

IDENTIFICATION

MEASUREMENT

ACTION

PROJECTION



Senior AI/ML Engineer - Computer Vision Specialty

- Experience working in a specialized Computer Vision role within the last 2 years required.
- Experience with working in a collaborative CI/CD software development environment, including use of git.
- Experience with developing deep learning models for diverse computer vision tasks (e.g., segmentation, recognition, classification, domain adaptation) using PyTorch, Tensorflow, or other deep learning frameworks.
- Experience in machine learning, software engineering, and Python.
- Experience with one or more open-source computer vision libraries such as Pillow, scikit-image, and OpenCV.

Analyze, design, code, and test multiple components of application code across one or more clients for enterprise-level identity management software. Perform maintenance, enhancements and/or development work. Must be proficient developing software with Python programming language, experience developing algorithms and software on Linux and Windows, and familiar with image processing concepts, file formats, and tools and python packages like PIL/Pillow and Scikit-Image

Required: PyTorch, Tensorflow, Keras, Numpy, OpenCV/Pillow.
Additionally Preferred: Scikit-Learn, Pandas, etc.



OpenCV and Image Processing Engineer

Computer Vision Engineer

Requirements:

- Strong experience with OpenCV and image processing algorithms
 - Proficient in Python
 - Experience with image processing libraries such as OpenCV, Scikit-image, scikit-learn, Tensorflow, Pytorch, Keras.
 - Strong understanding of computer vision concepts such as feature detection, object recognition, and image segmentation
-
- Passion or Experience in Computer Vision / Machine Learning technologies and tools
 - Experience with computer vision tools like OpenCV, YOLO, PIP, Tensorflow, PyTorch . Experience with Deep Learning, Neural Networks and Artificial Intelligence
 - Technical understanding of computer vision and machine learning
 - Familiar with state-of-the-art CV/ML models and tools



Most Popular Python Libraries for Computer Vision



CONTENTS

Most Popular Python Libraries for Image Processing

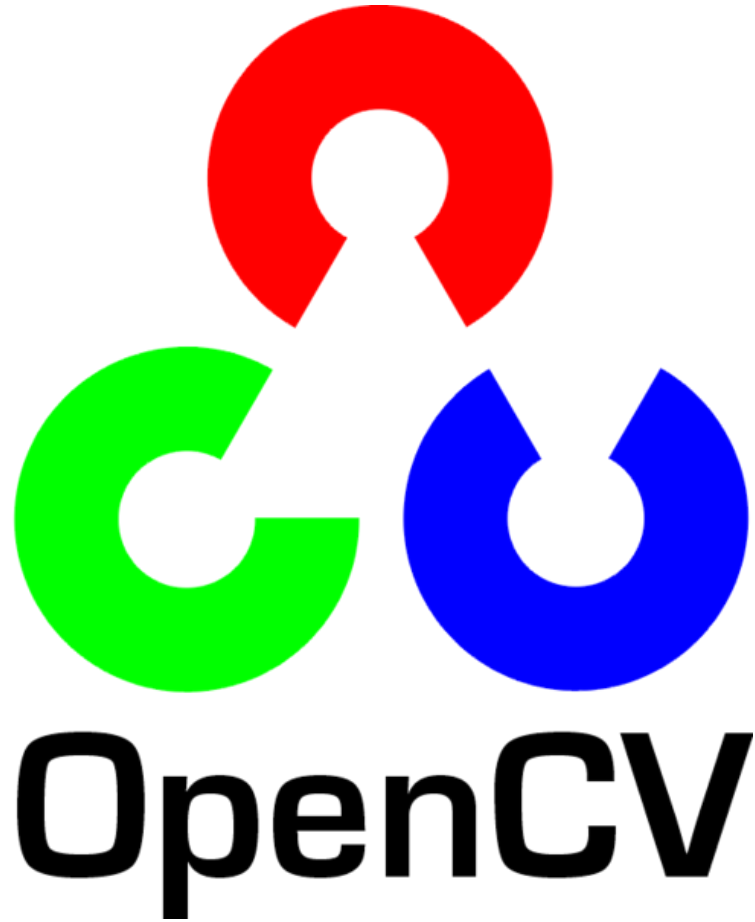
 Pillow

 Scikit-image

 OpenCV



Most Popular Python Libraries



pillow



scikit-image
image processing in python



pillow



Pillow

Python Imaging Library



PILLOW

- **Pillow** (Python Imaging Library) is a Python library used for **reading, writing, and manipulating** image files.
- It is an **enhanced version** of the PIL library, providing **basic image-processing** functionalities.
- **Pillow** is one of the **most popular** image-processing libraries for Python and is used in fields such as **computer vision, image processing, data analysis, machine learning, and deep learning**.
- **PIL** (Python Imaging Library) is the predecessor of **Pillow**.
- **Pillow** is a **fork** of PIL and has **replaced** PIL since development of PIL ceased. Therefore, while PIL and **Pillow** have **similar functionalities**, they are **technically different libraries**.



Why Rotate, Invert, Blur ..

These operations are generally performed for purposes such as image preprocessing, data augmentation, feature extraction, and model improvement.

- **Rotate:** Data augmentation + Recognizing objects from different angles
- **Invert (Color Inversion):** Data augmentation + Feature extraction
- **Blur:** Noise reduction + Data augmentation + Edge detection



PILLOW

- **Pillow** provides various functions for performing **image operations**. These operations include **resizing**, **cropping**, **rotating**, **scaling**, **flipping**, **filtering**, **color transforming**, and **converting** images to different file formats.
- **Pillow** collaborates with several high-performance C libraries such as **libjpeg**, **libtiff**, **libpng**, **libwebp**, **talloc**, and **Little CMS**.
- Additionally, **Pillow** provides interfaces for **image processing** tasks such as **creating various effects on images**, **transitioning between images**, and **merging images**. These interfaces can provide the foundational building blocks for any image-processing program.



DATE : 05.08.2025

DT/NT : DT

LESSON : DEEP LEARNING

**SUBJECT: COMPUTER VISION (CV) II –
SCIKIT IMAGE & OPENCV**

BATCH : B287

**DATA
SCIENCE**



TECHPRO
EDUCATION



techproeducation.com



+1 (585) 304 29 59





scikit-image



scikit-image
image processing in python



SCIKIT-IMAGE

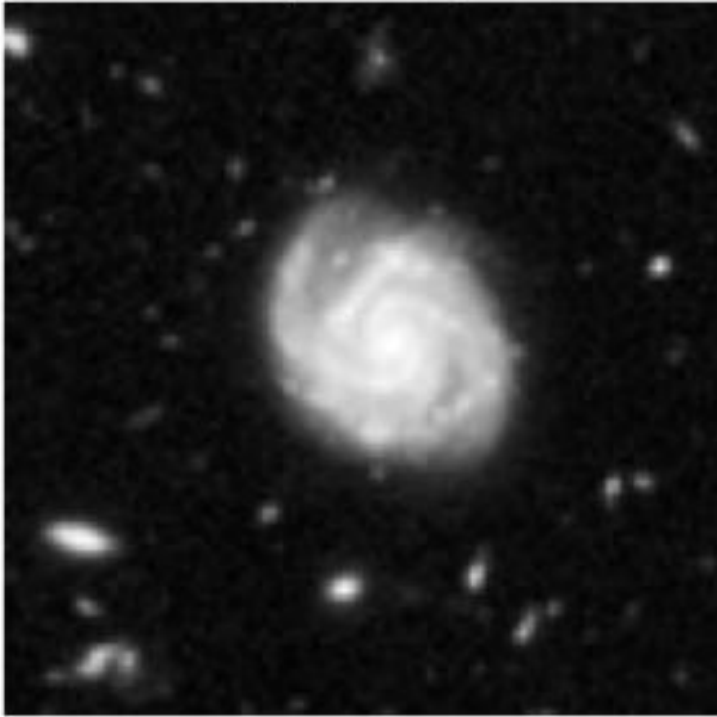
Scikit-image is a library used for **scientific image processing** in **Python**. This library, along with **NumPy**, **SciPy**, **Matplotlib** libraries, and other Python libraries, provides various tools for processing **2D and 3D images**. **Scikit-image** offers functions for many common operations used in image processing, including:

- **Image enhancement** (e.g., filtering, classification, segmentation, morphological operations)
- **Image transformations** (e.g., scaling, rotating, flipping, shifting)
- **Feature extraction** (e.g., edges, corners, lines, points)
- **Pattern recognition** (e.g., object detection, face recognition, tracking moving objects)
- **Numerical feature measurement** (e.g., intensity, surface area, volume)
- **3D image processing** (e.g., volume visualisation, volume segmentation)
- **Scikit-image** supports many common data formats, including TIFF, JPEG, PNG, BMP, FITS, etc.

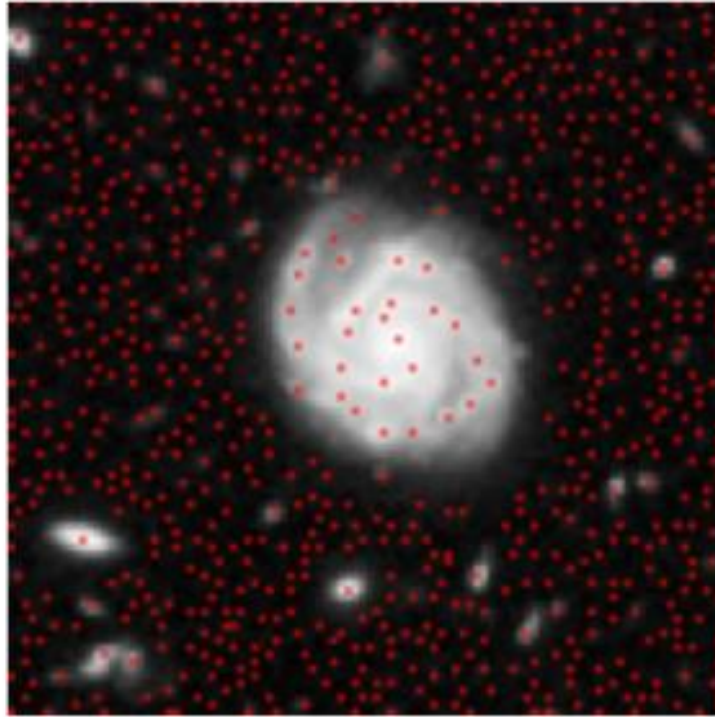


SCIKIT-IMAGE

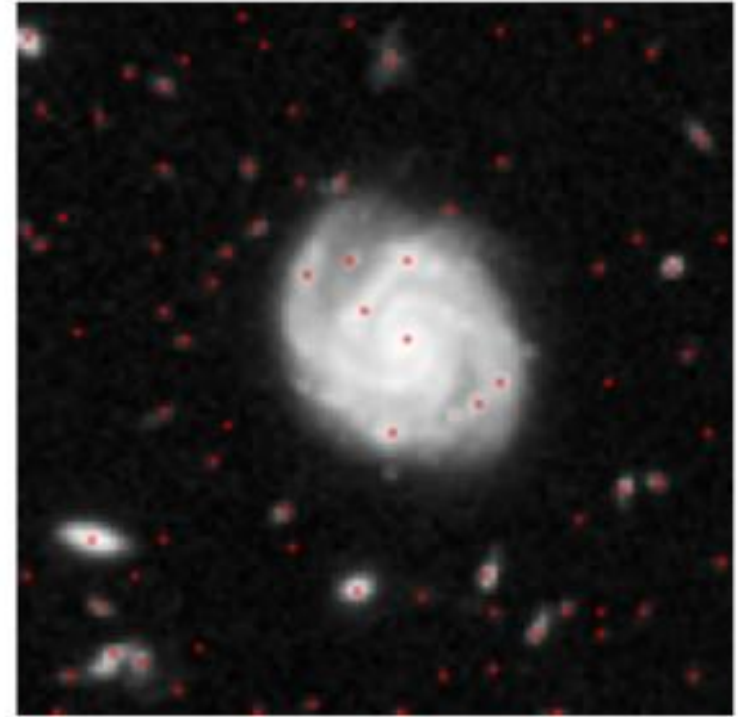
Original image



Local Maxima



h maxima for $h = 0.05$





SCIKIT-IMAGE

Scikit-image is an **image processing library** for the **Python** programming language, widely used in **Computer Vision**. It includes many functions that operate on **NumPy arrays**.

Critical features of **Scikit-image** include:

- Support for common **image processing operations** such as **filtering, thresholding, edge detection, Hough transformation**, and **feature extraction**.
- Capability to perform various **input/output operations** such as **image saving, loading, outputting**, and **resizing**.
- Compatibility with different **image formats** (JPEG, PNG, TIFF, etc.).
- Widely used in **scientific research** for supporting common image processing tasks.
- Integration with the **Python ecosystem** and compatibility with other libraries like **NumPy, Pandas, Matplotlib**, etc.

In summary, **Scikit-image** is a Python library that facilitates many common **image-processing operations** in **computer vision**.



Stable ([release notes](#))
0.20.0 - February 2023

[Download](#)

Development
pre-0.21

[Download](#)

- [GitHub](#) *source & bug reports*
- [Contribute](#) *get involved*
- [Get Help](#) *advice & community*
- [Discuss](#) *contributor talk*
- [StackOverflow](#) *coding help*

Image processing in Python

scikit-image is a collection of algorithms for image processing. It is available **free of charge and free of restriction**. We pride ourselves on high-quality, peer-reviewed code, written by an active **community of volunteers**.

[Download](#)

If you find this project useful, please cite: [\[BiBTeX\]](#)

Stéfan van der Walt, Johannes L. Schönberger, Juan Nunez-Iglesias, François Boulogne, Joshua D. Warner, Neil Yager, Emmanuelle Gouillart, Tony Yu and the scikit-image contributors. **scikit-image: Image processing in Python**. PeerJ 2:e453 (2014)
<https://doi.org/10.7717/peerj.453>

<https://scikit-image.org/>



SCIKIT-IMAGE

Original image



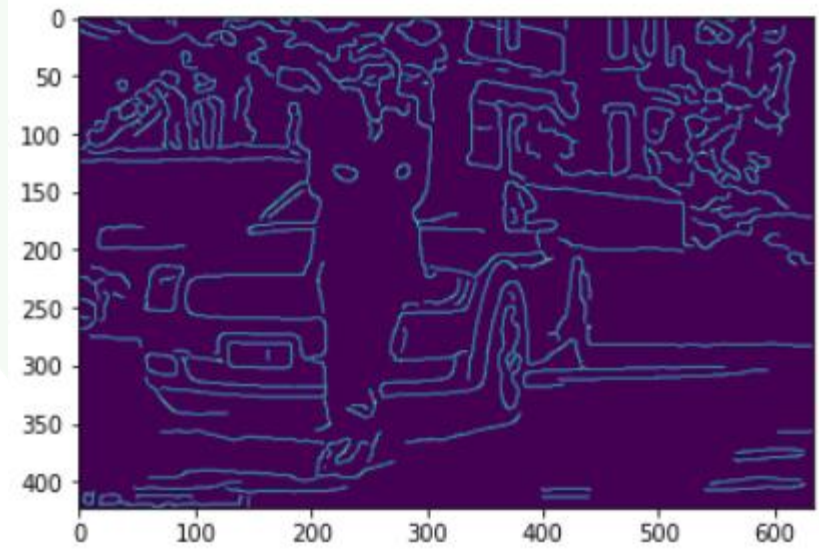
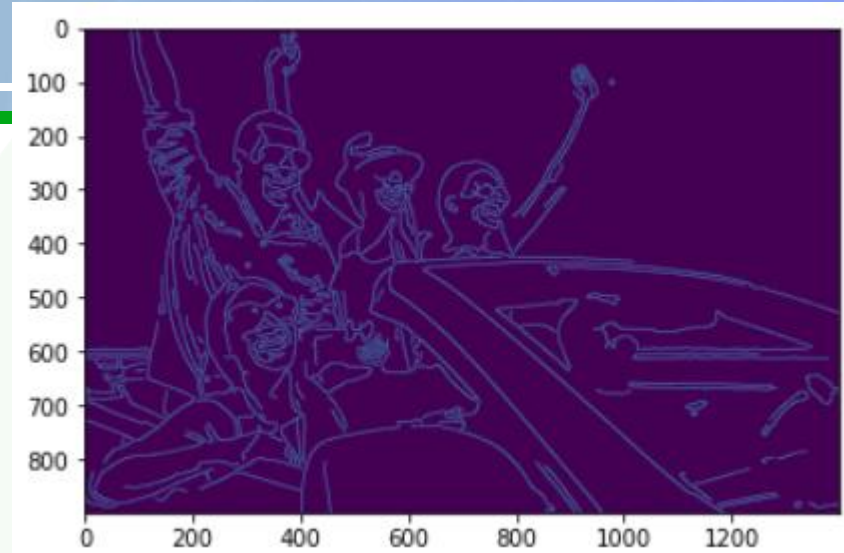
Roberts Edge Detection



Sobel



Scharr





OpenCV



https://www.youtube.com/shorts/bJ_22WjW_tg

Feature/Criteria	Pillow (PIL)	scikit-image	OpenCV
Purpose	Basic image processing, reading, writing, converting	Scientific image processing, analysis, and feature extraction	Comprehensive image processing and computer vision
Ease of Use	Very easy and Pythonic	Easy and understandable	Extensive API, can be complex for beginners
Supported Formats	JPG, PNG, BMP, GIF, TIFF, etc.	NumPy array, many file formats	Very extensive (JPG, PNG, BMP, TIFF, video formats)
Core Functions	Open, save, crop, rotate, resize, simple filters	Segmentation, morphological operations, feature extraction, measurements, advanced filters	All basic functions, advanced operations, video processing, machine learning
Color Modes	RGB, RGBA, L (grayscale), etc.	Generally RGB/gray with NumPy array	RGB, BGR, grayscale, YUV, HSV, and multiple supports
Filters and Effects	Simple filters (blur, sharpen, blend, mask)	Advanced filters (edge detection, histogram, morph, denoising)	Very advanced filters, edge, contour, pattern recognition
Video Processing	No	No	Yes (video read/write, frame analysis)



Feature/Criteria	Pillow (PIL)	scikit-image	OpenCV
Machine Learning / CNN	No	No (but integrates with NumPy)	Built-in machine learning, CNN, DNN modules
Hardware Support	No	No	GPU acceleration (OpenCL, CUDA)
Documentation and Community	Extensive and simple	Common in the scientific community	Very large, worldwide massive community
Platform Support	Python only	Python only	Python, C++, Java, Android, iOS, etc.
Library Size	Lightweight	Medium	Large (contains many modules)
Typical Use Cases	Rapid prototyping, simple visual processing	Scientific research, image analysis	Industry, AR/VR, real-time applications, video



OpenCV



OpenCV

Open Source Computer Vision Library



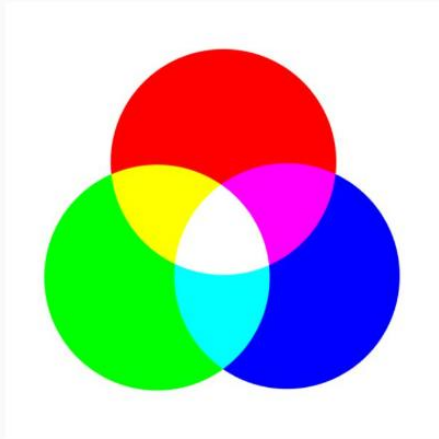
OpenCV

- **OpenCV** is an **open-source image processing** library. Initially developed by **Intel in 1999**, it has since been supported by companies and communities including **Itseez, Willow, Nvidia, AMD, and Google**.
- As of **May 27, 2016**, OpenCV's development was continued under **Intel's ownership** after Intel acquired Itseez.
- The library is **platform-independent**, capable of running on **Windows, Linux, FreeBSD, Android, Mac OS, and iOS**, and supports development in nearly all programming languages.
- Within the OpenCV library, there are over **2500 algorithms** dedicated to **image processing** and **machine learning**, facilitating tasks such as **face recognition, object differentiation, human motion detection, object classification, license plate recognition, 3D image manipulation, image comparison, and Optical Character Recognition (OCR)**, among others.



OpenCV

RGB - Red Green Blue



BGR color profile



RGB color profile

OpenCV uses **BGR color space** instead of RGB for images. This **is because OpenCV** was first **developed for Intel** processors. **Intel processors** write data to memory using **BGR color space**, so OpenCV also uses BGR color space.

This should be taken into consideration when working with OpenCV.



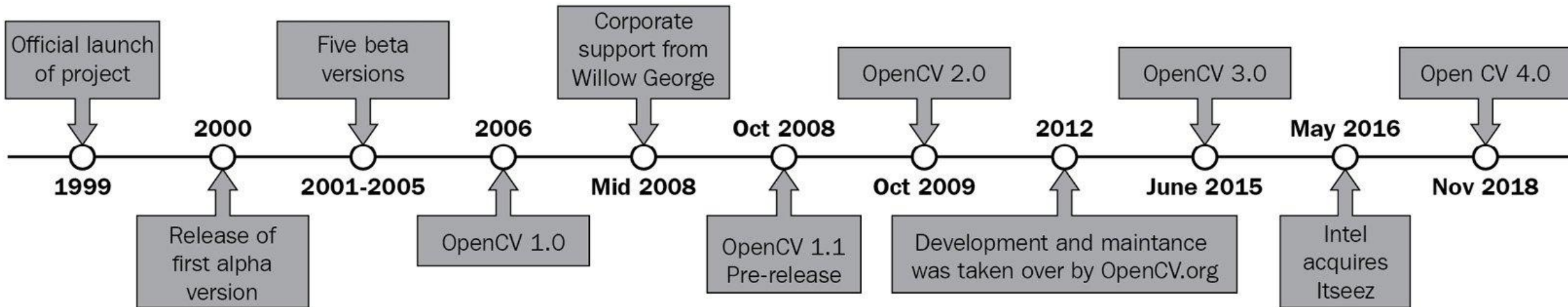
What is OpenCV?

- ❑ OpenCV stands for Open Source Computer Vision Library
- ❑ Being developed at Intel since 1999
- ❑ Written in C/C++; Contains over 500 functions.
- ❑ Available on Windows, Linux and MacOSX.
- ❑ So far is extensively used in many companies and research centers



OpenCV

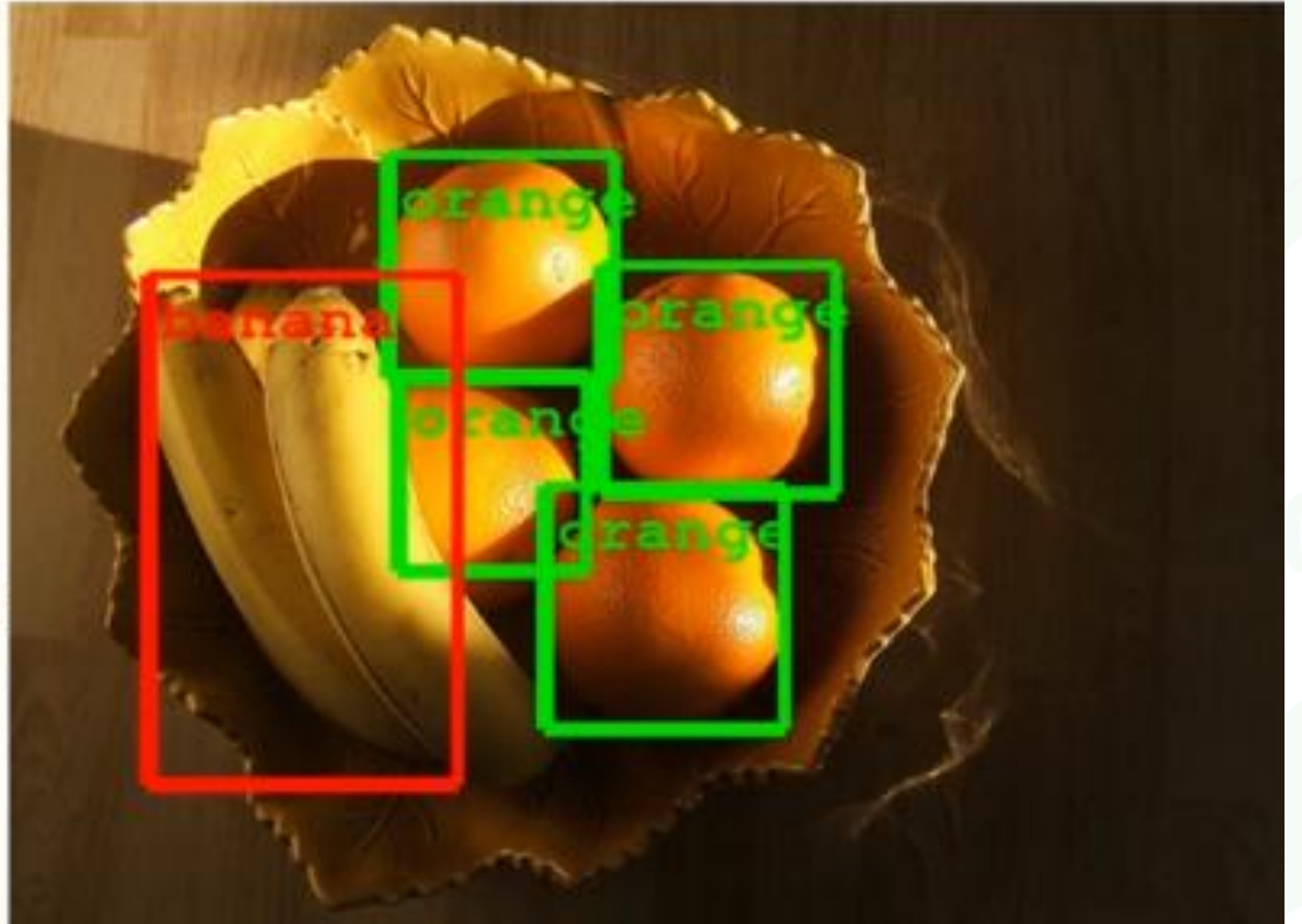
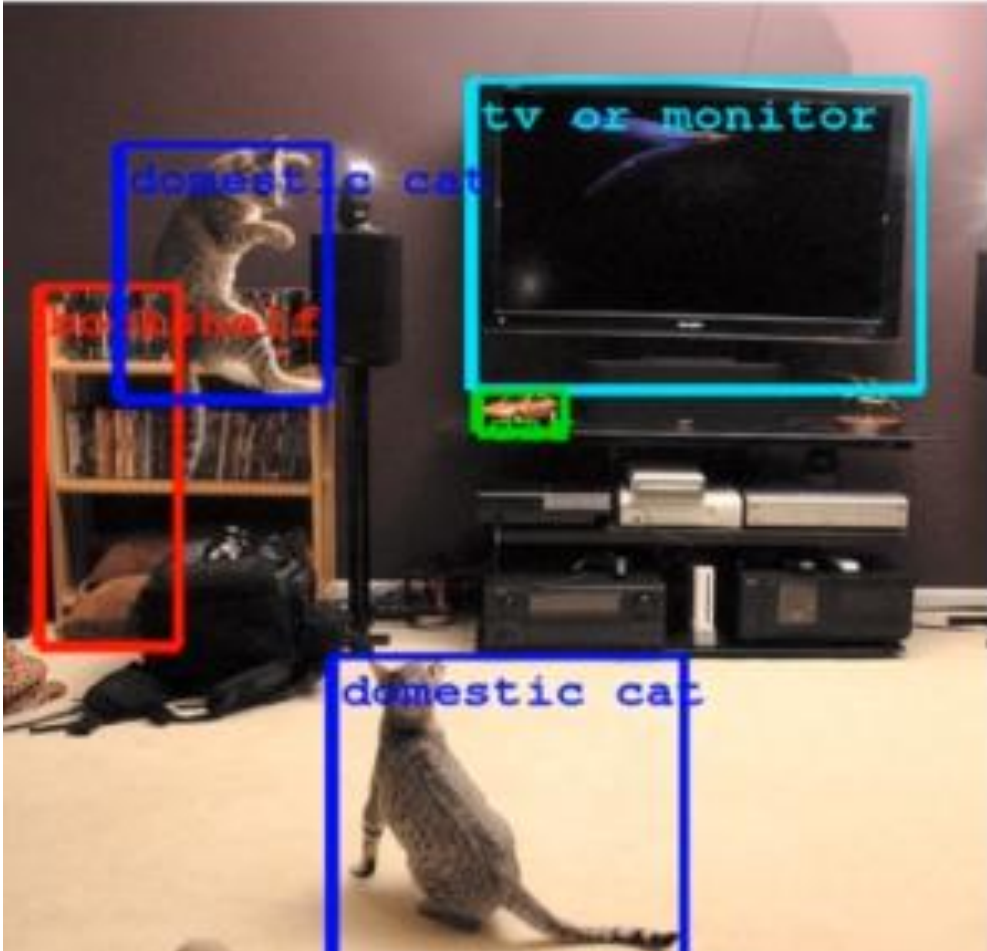
Timeline of OpenCV





OpenCV

OpenCV is useful for object detection.





OpenCV

Applications of OpenCV

- Gesture recognition
- Human-computer interaction(HCI)
- Mobile robotics
- Segmentation and recognition
- Motion tracking
- Augmented reality
- Machine learning

<https://opencv.org/>



Free Courses ▾

Services

TensorFlow Keras Course **FREE**

OpenCV Course **FREE**

Python for Beginners **FREE**

TensorFlow Bootcamp

Learn how Neural Networks, Multi-Layer Perceptron and Convolutional Neural Networks are used for Image Classification and Segmentation, and train your own CNN model on a custom dataset. All this, in just 3 hours!

Course Code
TFBC

Type
Beginner

Available in
Python

Price
Free*



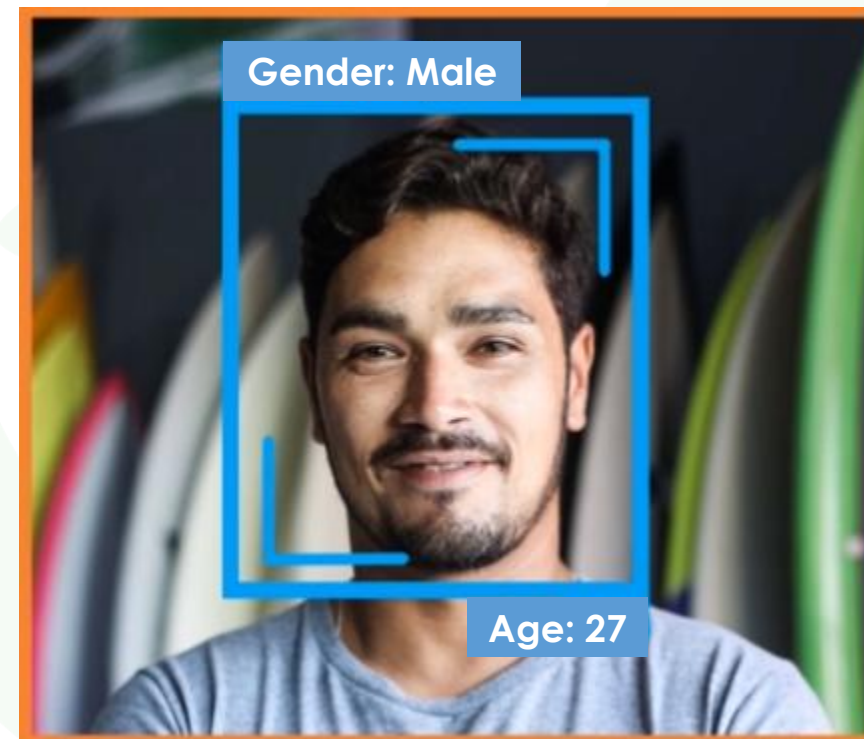
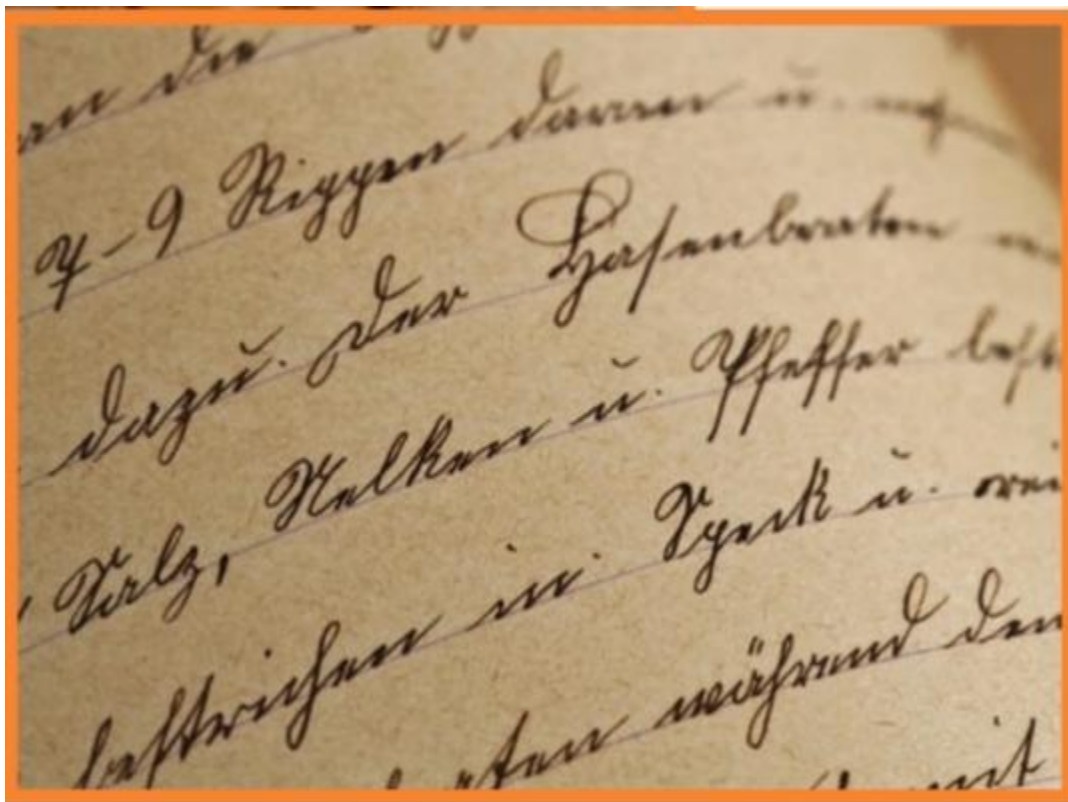


OpenCV



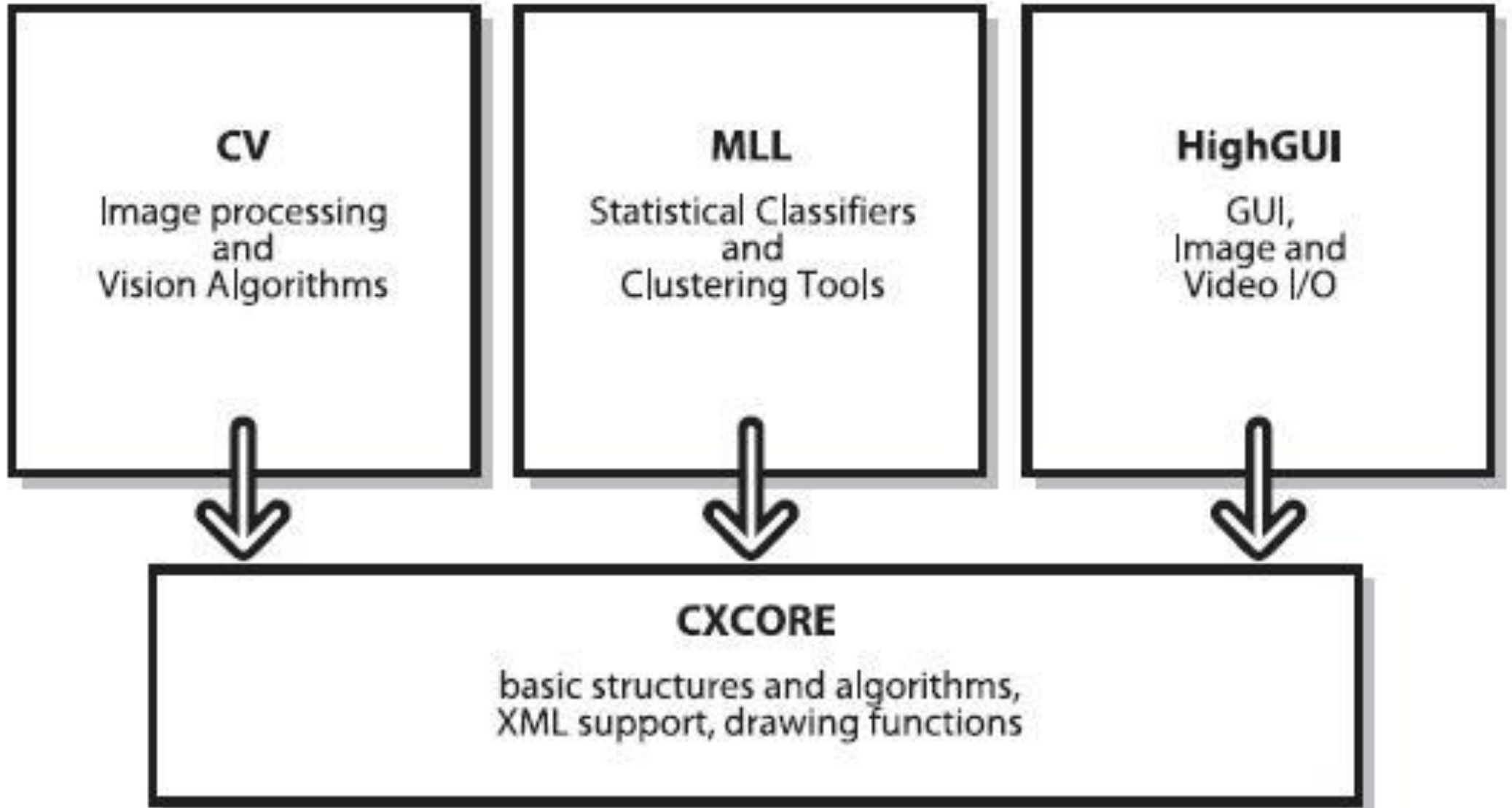


OpenCV





OpenCV





OpenCV

OpenCV's five main components:

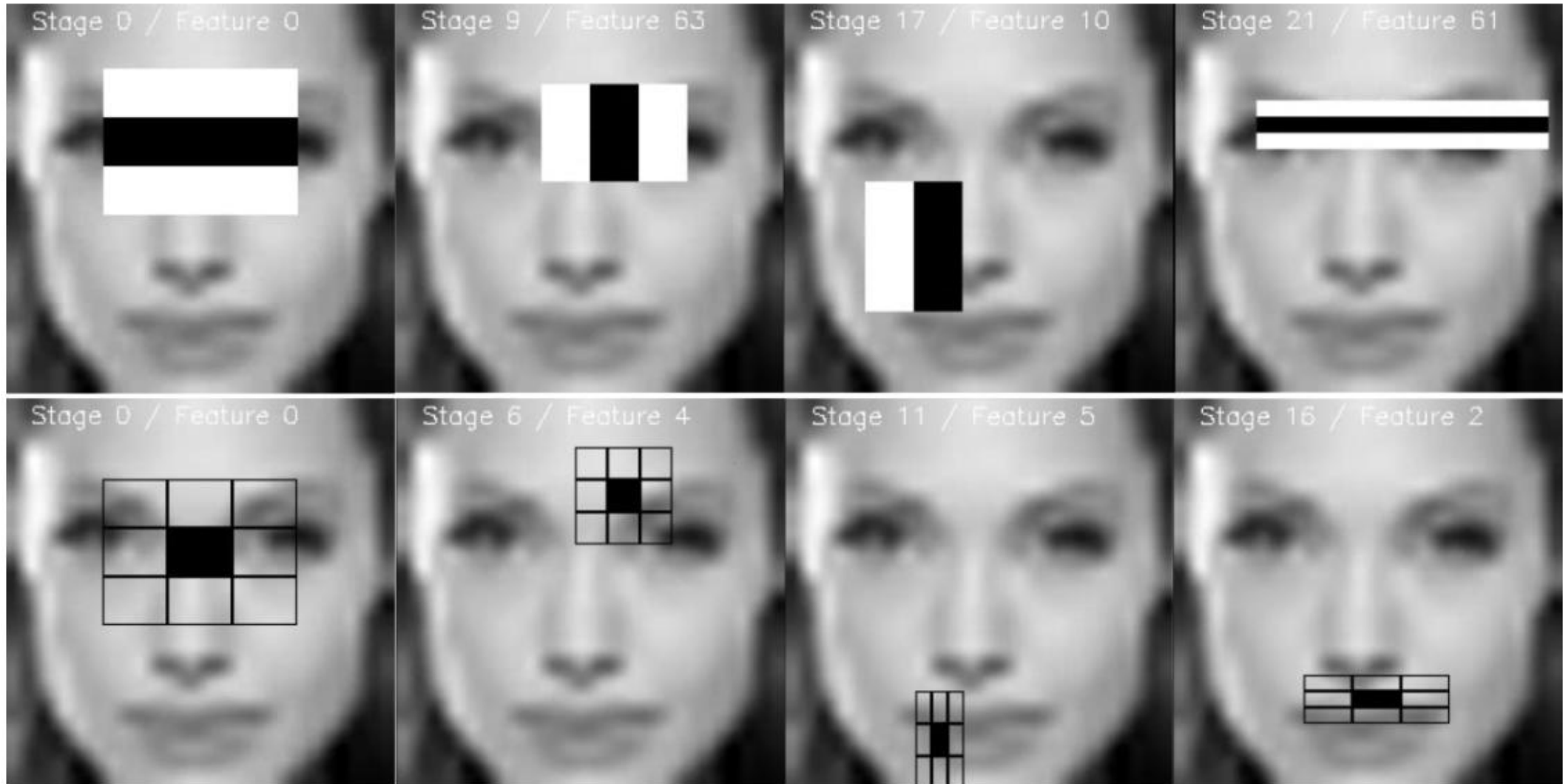
- 1. Core Component:** Includes fundamental **image processing** and **computer vision** functions. It handles tasks like matrix operations, reading/writing images, colour transformations, geometric transformations, image matching, object detection, and tracking.
- 2. Image Processing Component:** Focuses on various **image processing techniques** such as noise reduction, edge detection, histogram equalization, filtering, morphological operations, and image pyramids.
- 3. Video Processing Component:** Deals with **video processing** tasks including reading/writing videos, video compression, motion detection, and object tracking.
- 4. HighGUI Component:** Manages **graphical user interface (GUI)** operations for displaying images and videos and handling user interactions with mouse and keyboard.
- 5. Machine Learning Component:** Provides **machine learning** functionalities for tasks like classification, clustering, object recognition, and integrates with popular machine learning libraries starting from OpenCV version 4.0.



VIOLA-JONES



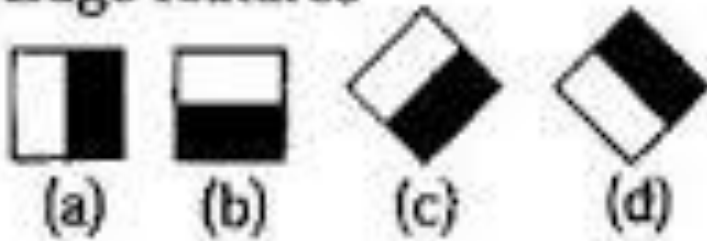
HAAR-LIKE FEATURES



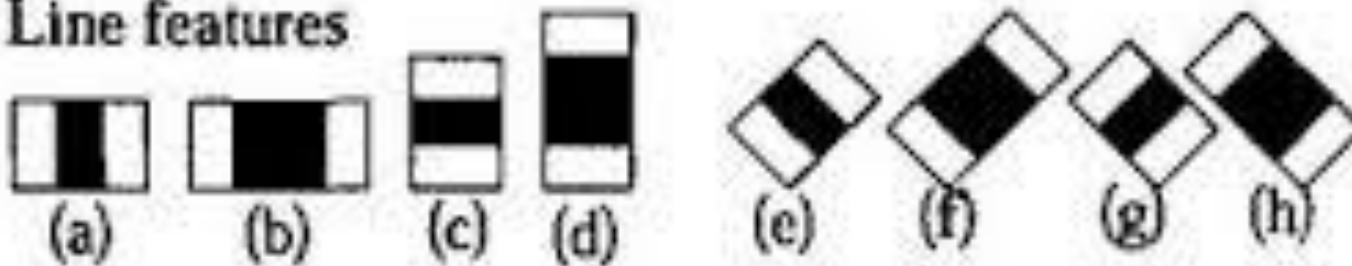


HAAR-LIKE FEATURES

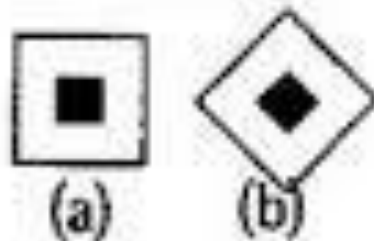
1. Edge features



2. Line features

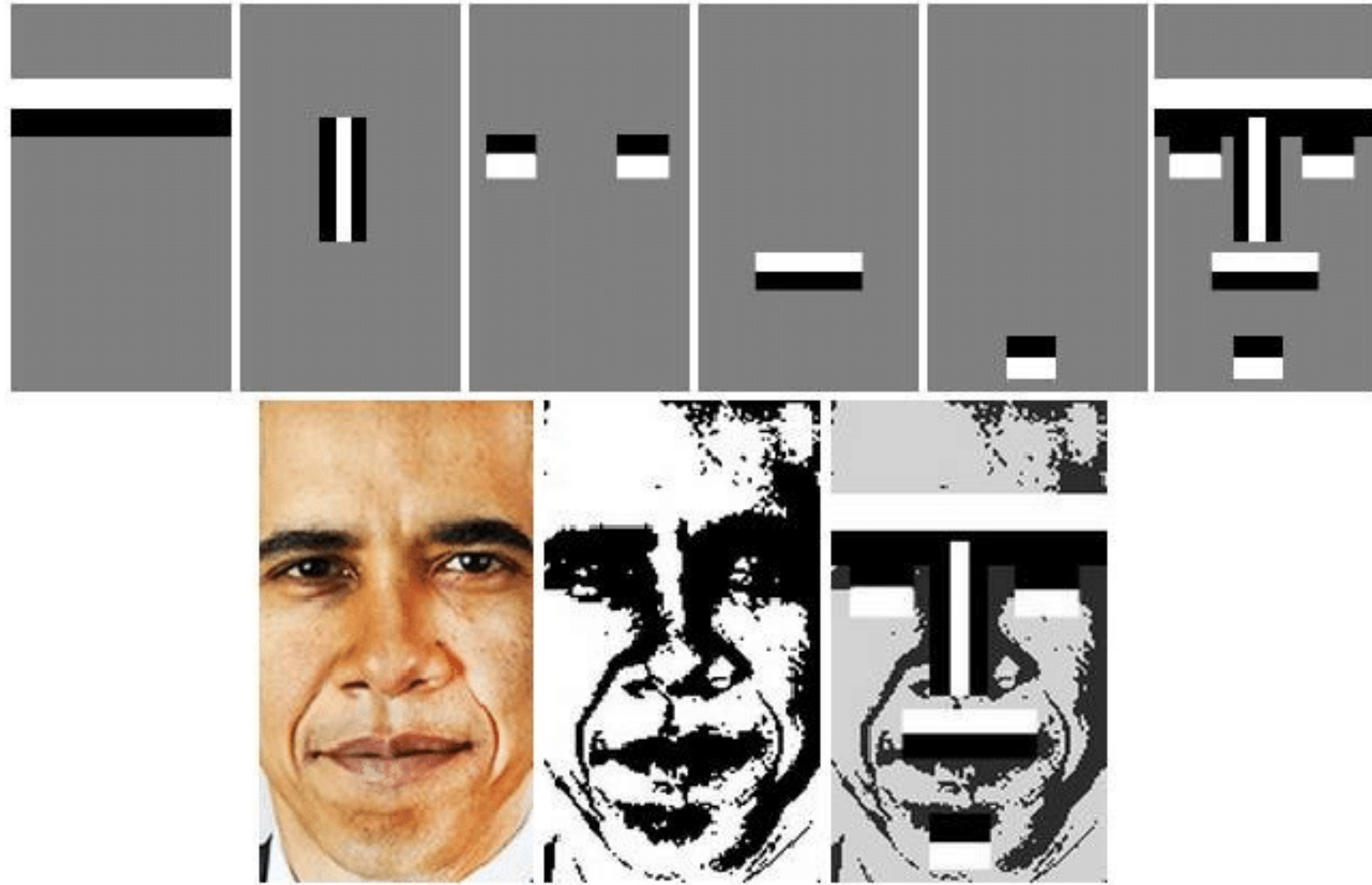


3. Center-surround features





HAAR-LIKE FEATURES





HAAR-LIKE FEATURES



DATE : 06.08.2025

DT/NT : DT

LESSON : DEEP LEARNING

SUBJECT: YOLO

BATCH : B287

**DATA
SCIENCE**



TECHPRO
EDUCATION



techproeducation.com



+1 (585) 304 29 59



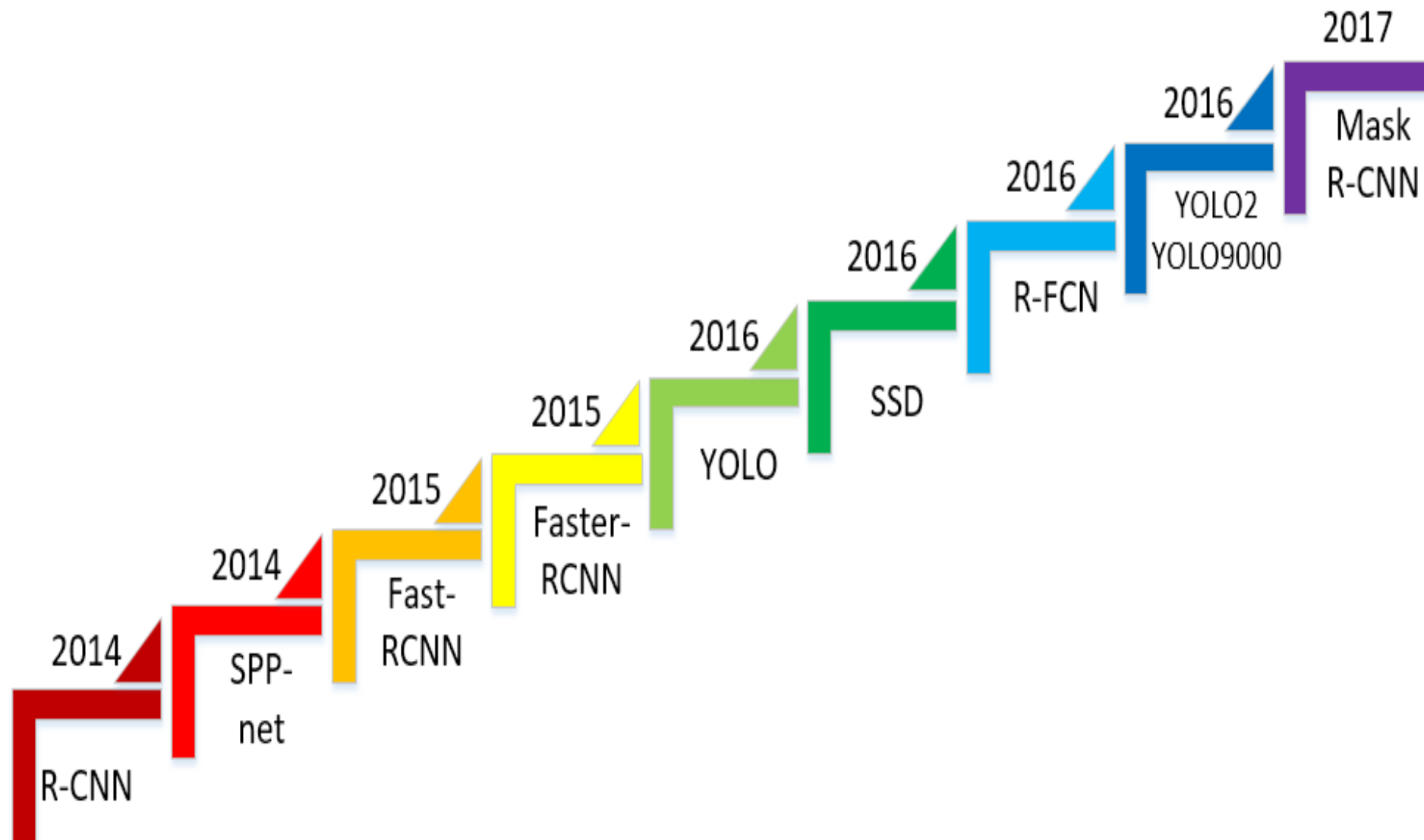
YOLO



YOLO



CNN



You Only Look Once: Unified, Real-Time Object Detection

Joseph Redmon*, Santosh Divvala*[†], Ross Girshick[‡], Ali Farhadi*[†]

University of Washington*, Allen Institute for AI[†], Facebook AI Research[‡]

<http://pjreddie.com/yolo/>

Abstract

We present YOLO, a new approach to object detection. Prior work on object detection repurposes classifiers to perform detection. Instead, we frame object detection as a regression problem to spatially separated bounding boxes and associated class probabilities. A single neural network predicts bounding boxes and class probabilities directly from full images in one evaluation. Since the whole detection pipeline is a single network, it can be optimized end-to-end directly on detection performance.

Our unified architecture is extremely fast. Our base YOLO model processes images in real-time at 45 frames per second. A smaller version of the network, Fast YOLO, processes an astounding 155 frames per second while still achieving double the mAP of other real-time detectors. Compared to state-of-the-art detection systems, YOLO makes more localization errors but is less likely to predict false positives on background. Finally, YOLO learns very general representations of objects. It outperforms other detection methods, including DPM and R-CNN, when generalizing from natural images to other domains like artwork.

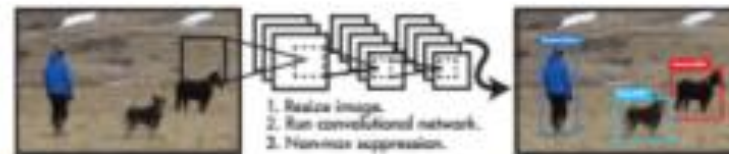


Figure 1: The YOLO Detection System. Processing images with YOLO is simple and straightforward. Our system (1) resizes the input image to 448×448 , (2) runs a single convolutional network on the image, and (3) thresholds the resulting detections by the model’s confidence.

methods to first generate potential bounding boxes in an image and then run a classifier on these proposed boxes. After classification, post-processing is used to refine the bounding boxes, eliminate duplicate detections, and rescore the boxes based on other objects in the scene [13]. These complex pipelines are slow and hard to optimize because each individual component must be trained separately.

We reframe object detection as a single regression problem, straight from image pixels to bounding box coordinates and class probabilities. Using our system, you only look once (YOLO) at an image to predict what objects are

YOLOv5



YOLOv7



YOLOv8

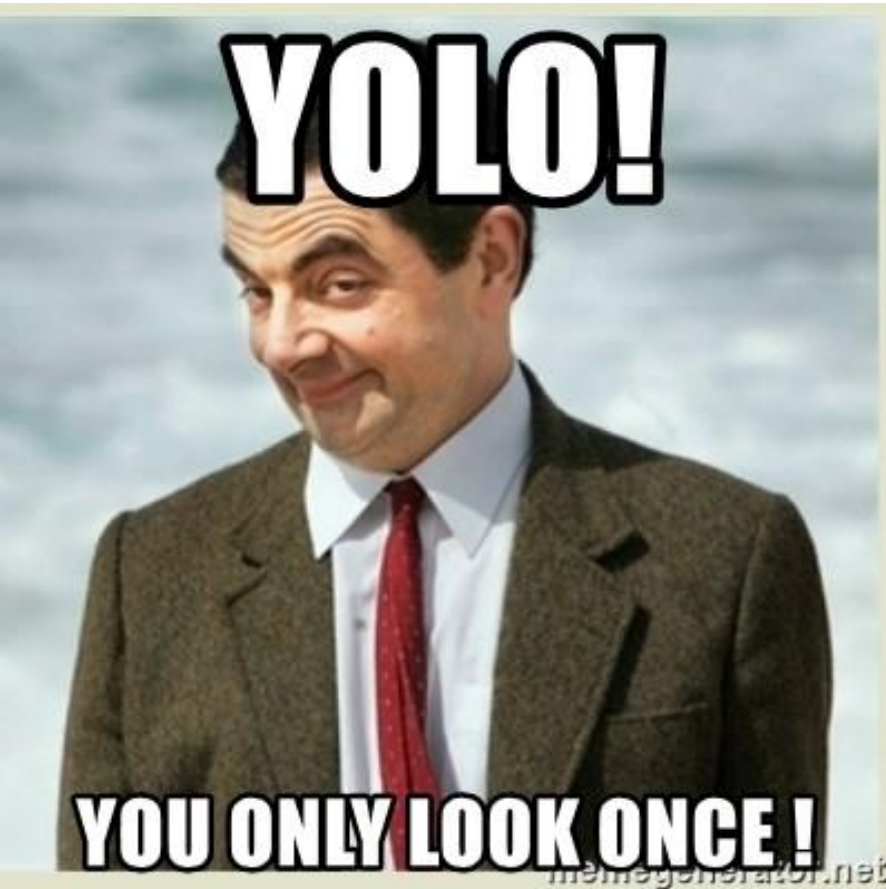


YOLO-NAS





YOLO



You Only Look Once

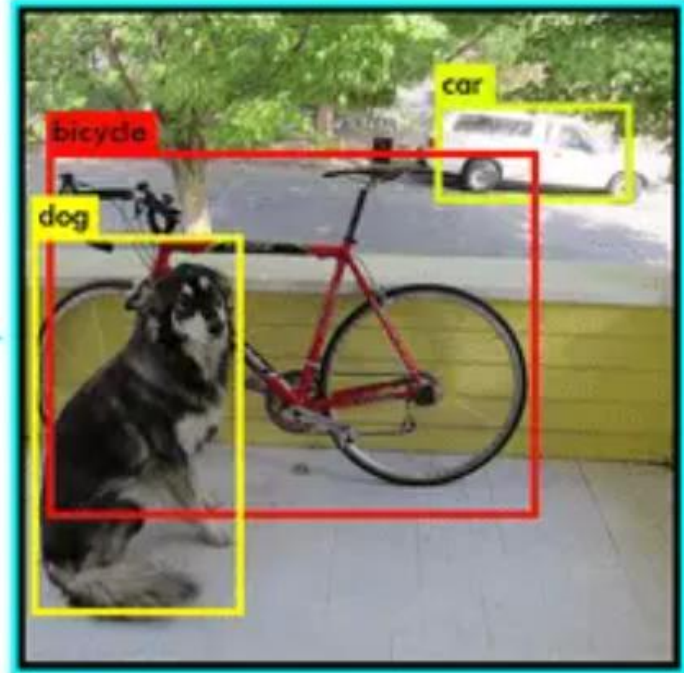
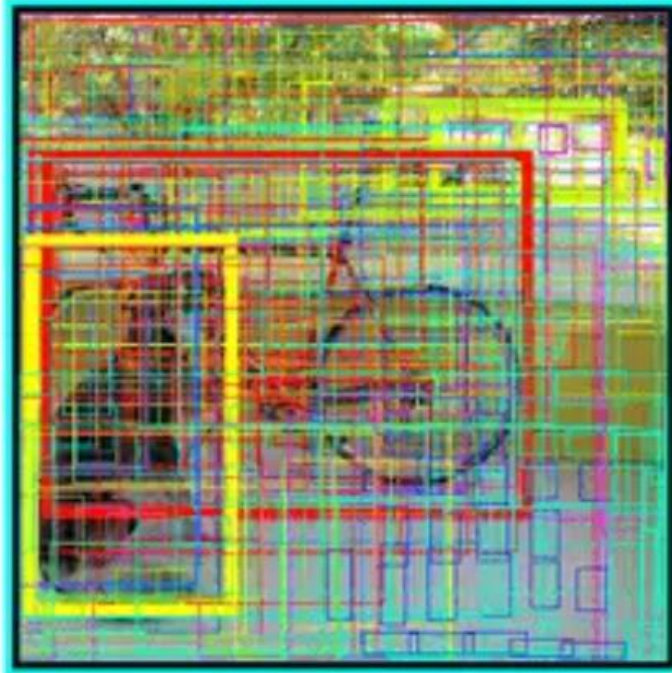
Object Detection

Real time detection



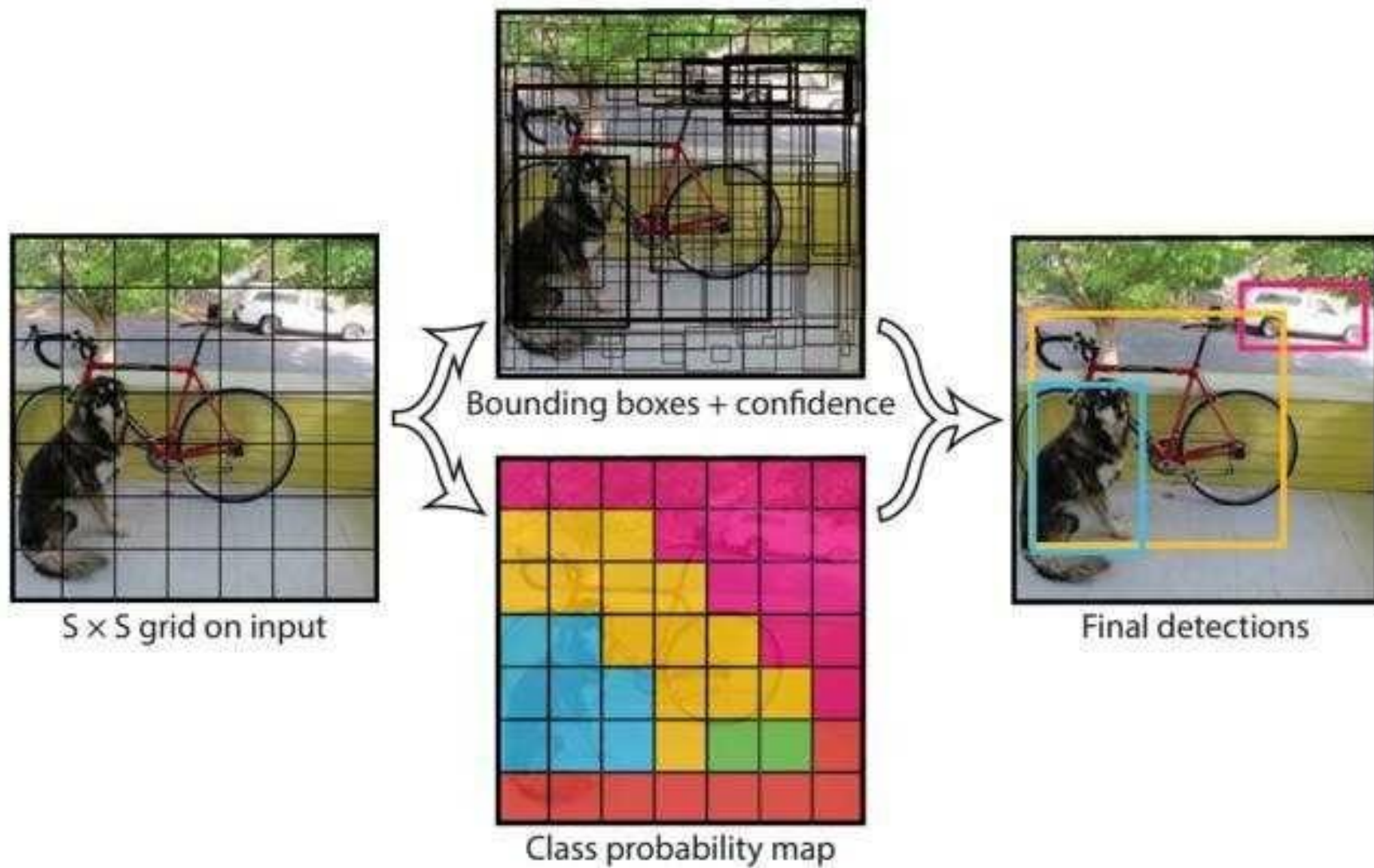


YOLO uses a grid instead of a filter.



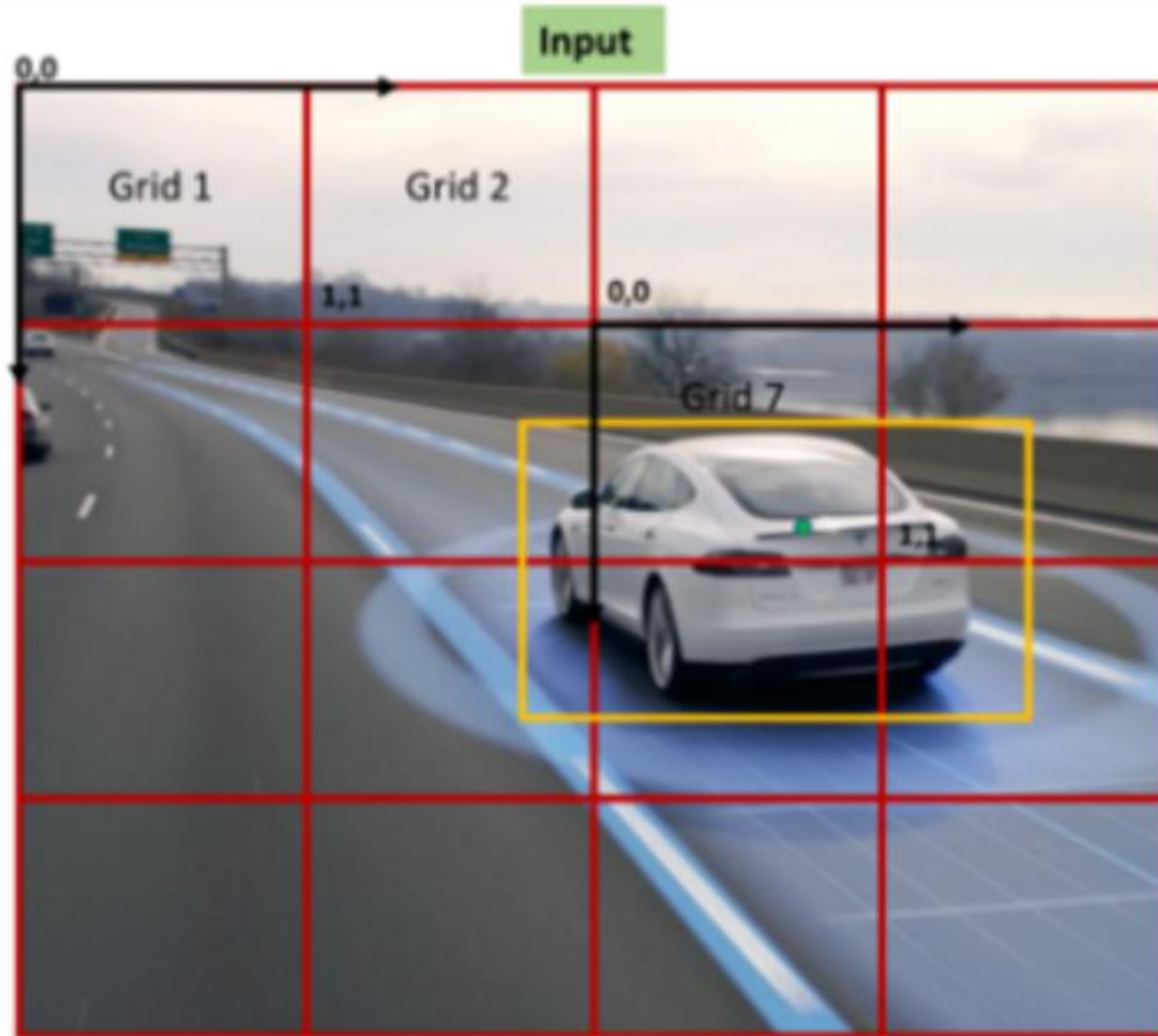


YOLO uses a grid instead of a filter.





YOLO uses a grid instead of a filter.





YOLO

Object Detection is the problem of locating and classifying objects in an image.

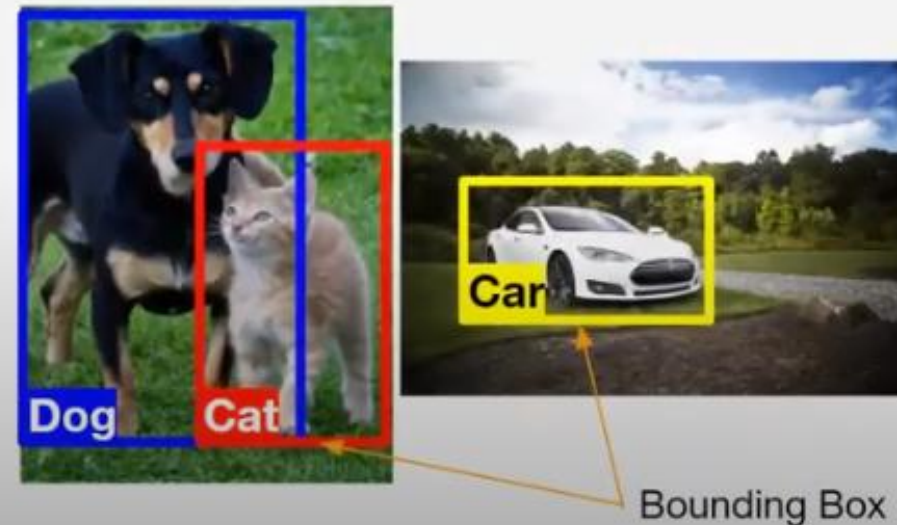
Classification

- One object and label per image



Object Detection

- Multiple objects per image
- Determine objects' location



While we can focus on an entire image in CNN, YOLO can focus directly on a bounding box, detecting more details.

Classify



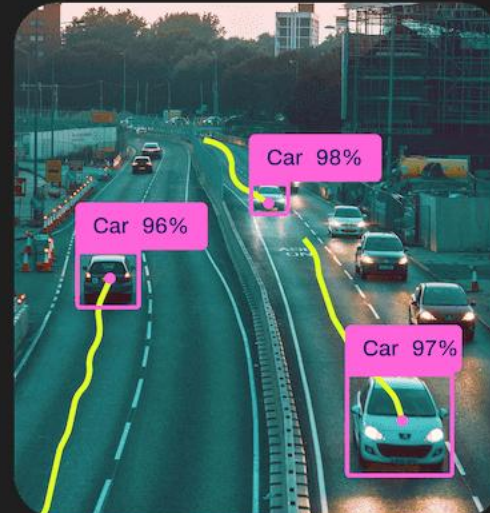
Detect



Segment



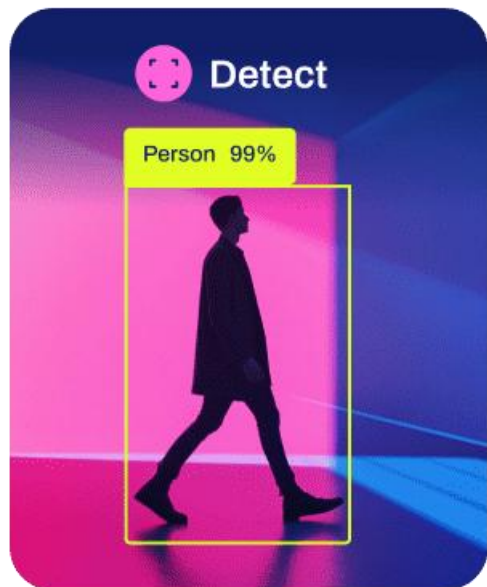
Track



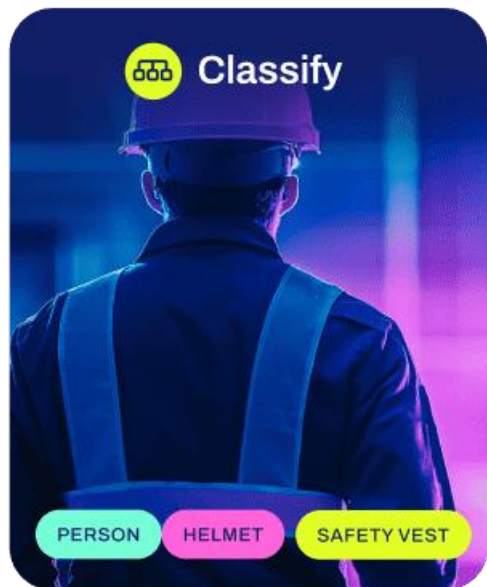
Pose



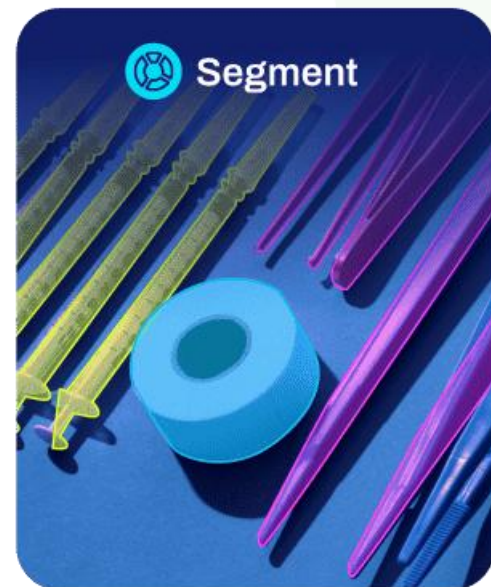
Detect



Classify



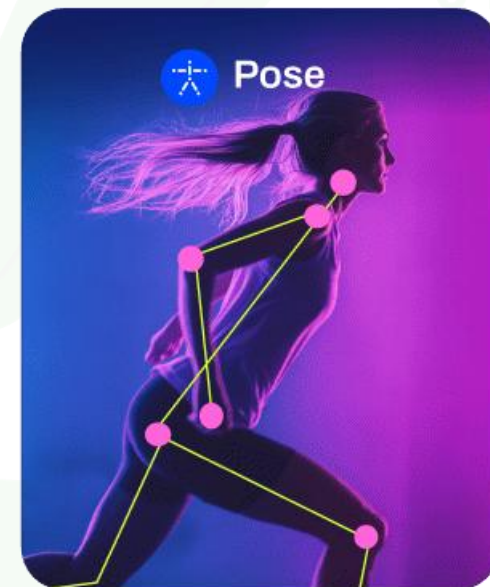
Segment



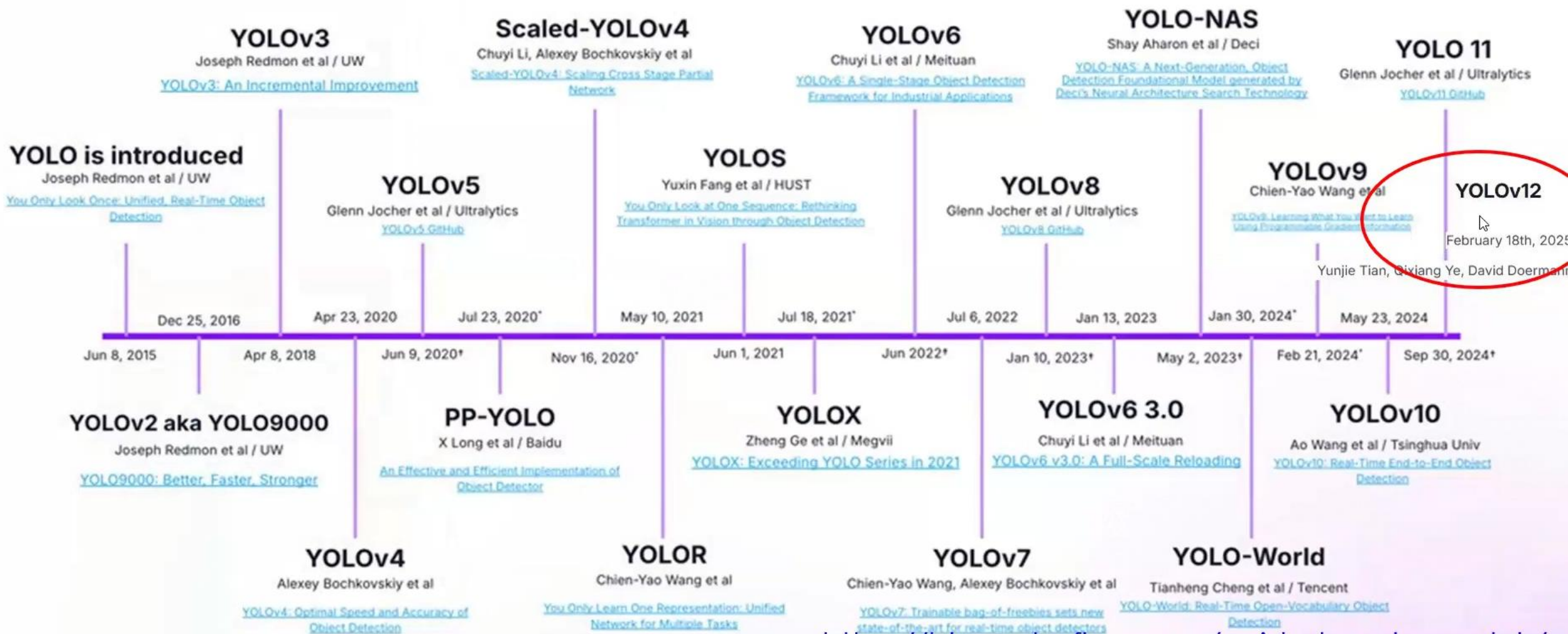
Track



Pose



YOLO Greatest Hits



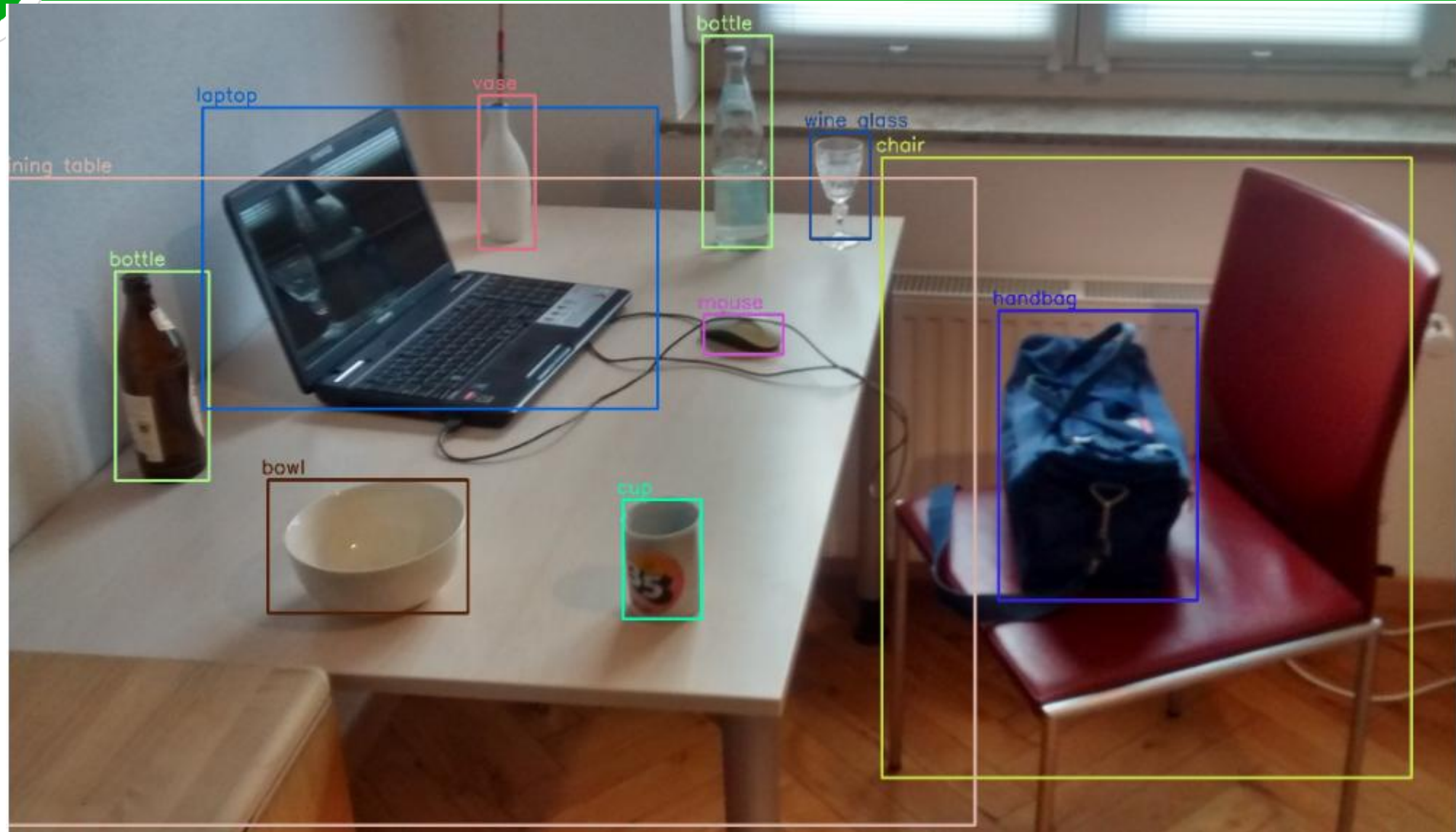
<https://www.ikomia.ai/blog/what-is-yolo-introduction-object-detection-computer-vision>

<https://blog.roboflow.com/guide-to-yolo-models/>

<https://docs.ultralytics.com/models/yolo11/>



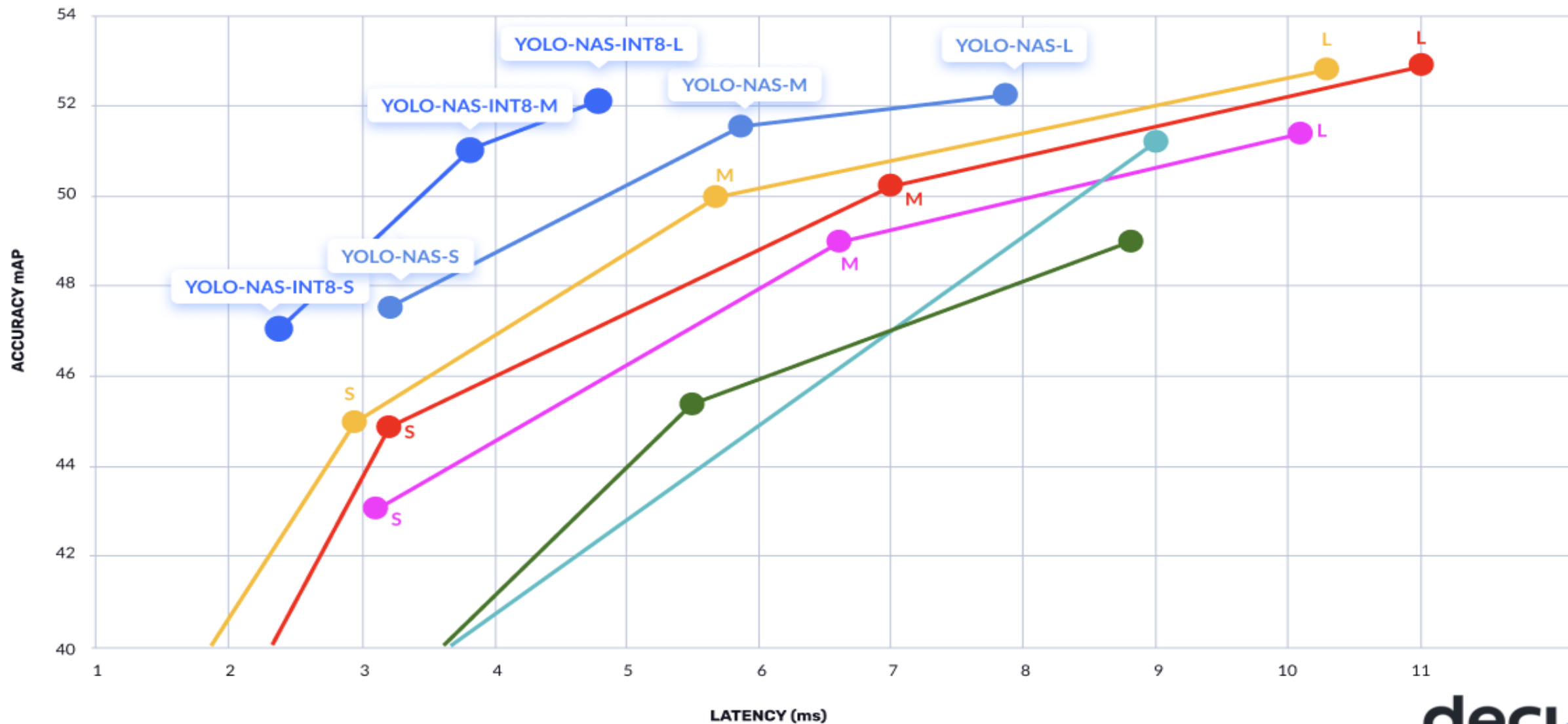
It uses bounding boxes to focus on details.



<https://www.labellerr.com/blog/why-is-the-yolo-algorithm-important/>

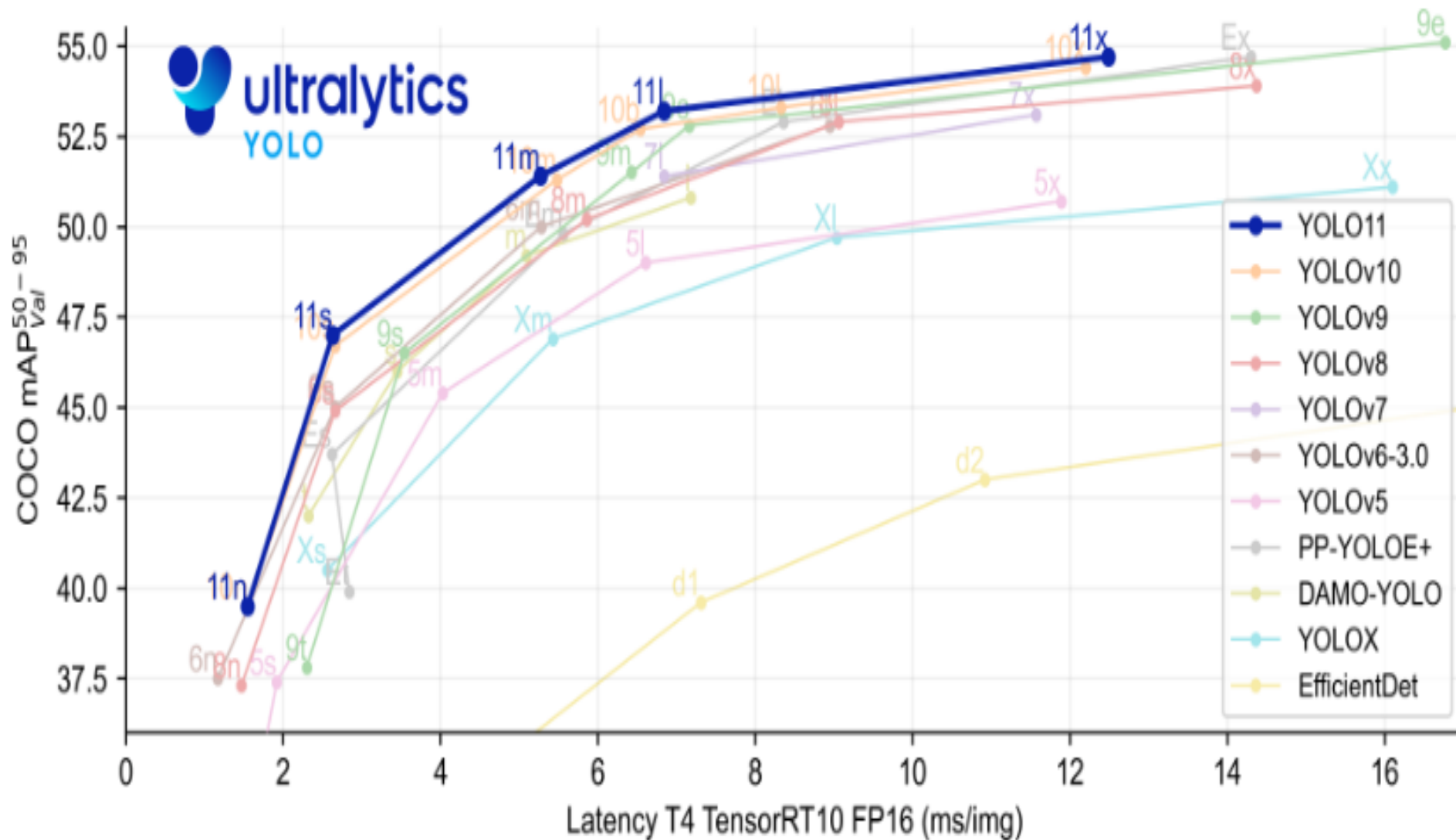
Efficient Frontier of Object Detection on COCO, Measured on NVIDIA T4

YOLO-NAS-FP16 YOLO-NAS-INT8 YOLO-V6-3.0 YOLO-V8 PPYOLOE YOLO-V7 YOLO-V5





YOLO 11



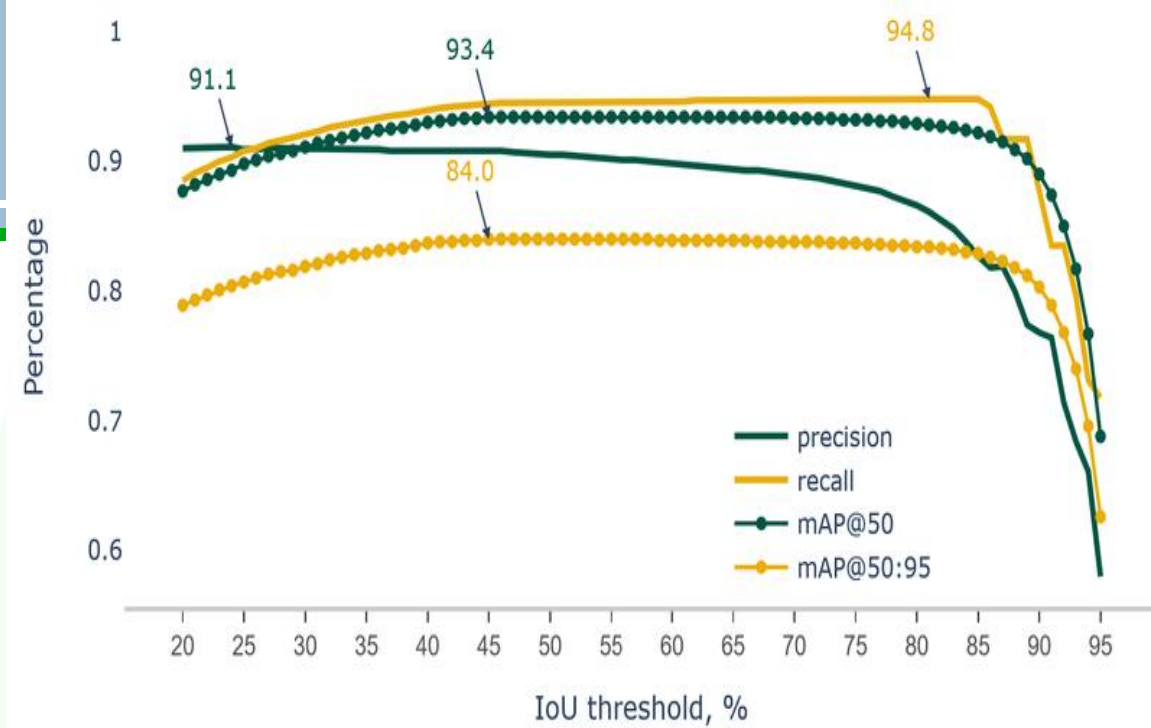


YOLO

We can measure the performance of YOLO models by using the following eval metrics:

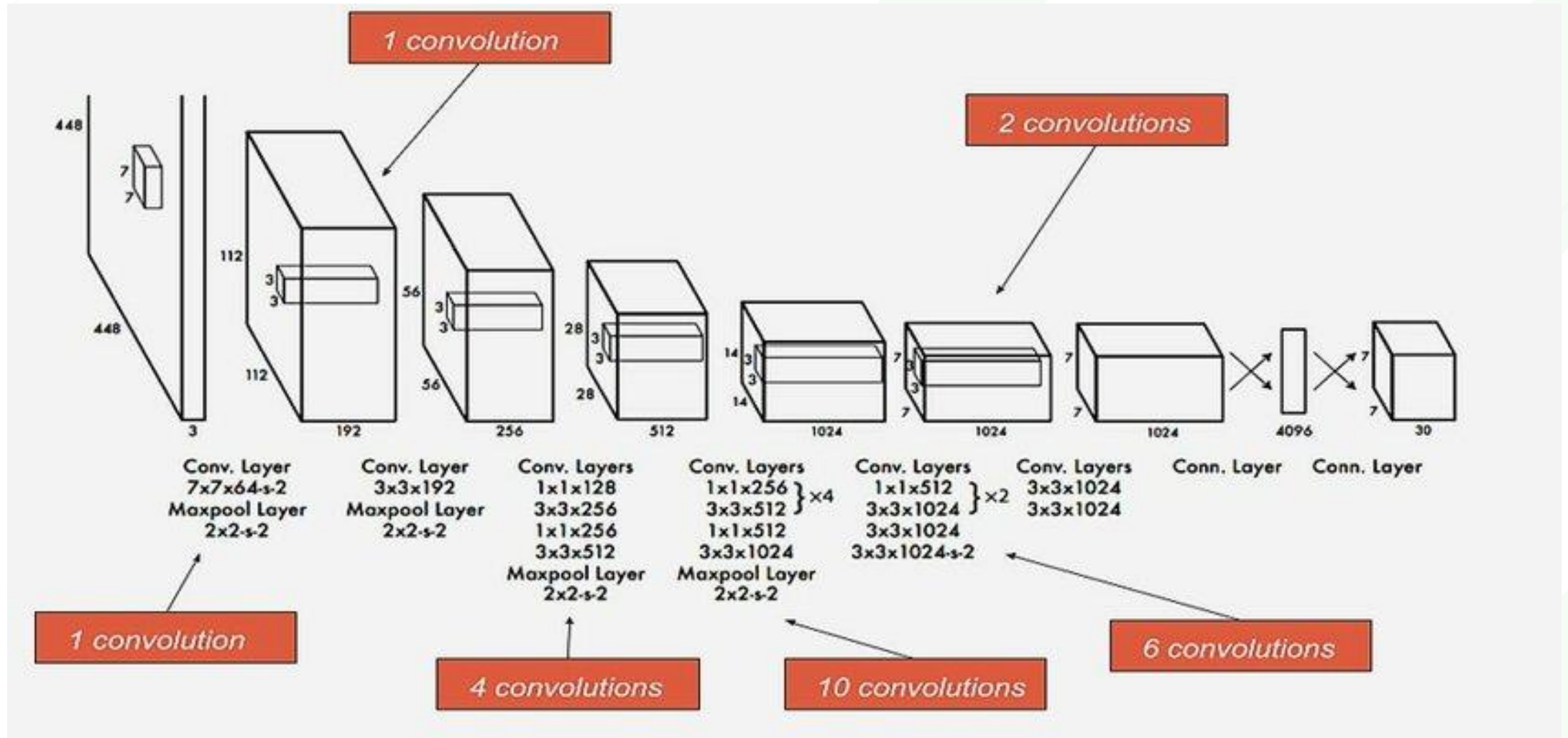
- **mAP (Mean Average Precision)** is a crucial metric used to measure and evaluate the accuracy and precision of object detection algorithms.
- **mAP@50** considers precision only at the top 50 predictions, focusing on a narrower range of predictions.
- **mAP@50:95** calculates precision across a wider range, from 50 to 95, providing a more comprehensive evaluation of the model's overall performance.
- Therefore, **mAP@50:95** offers a more detailed assessment compared to **mAP@50**, as it considers a broader range of predictions and their precision levels.

If these figures are close to the top left corner, that model is highly successful.





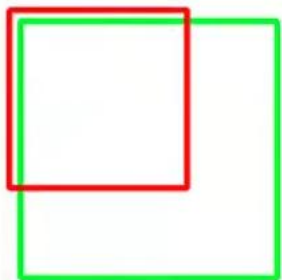
YOLO





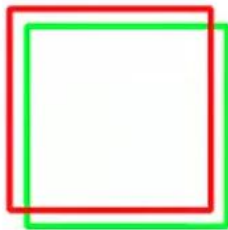
mAP50 – mAP95 - Intersection Over Union

IoU: 0.4034



Poor

IoU: 0.7330



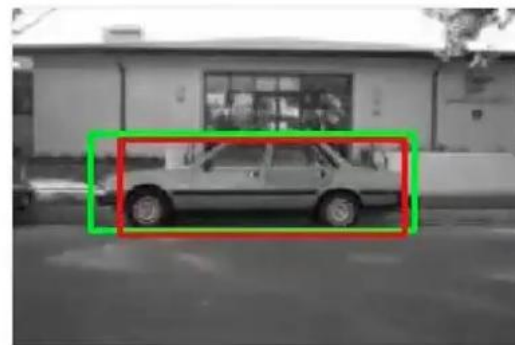
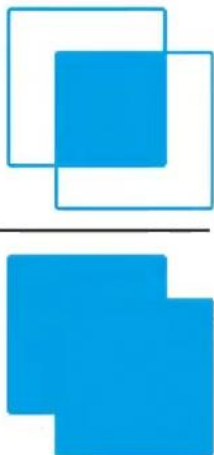
Good

IoU: 0.9264

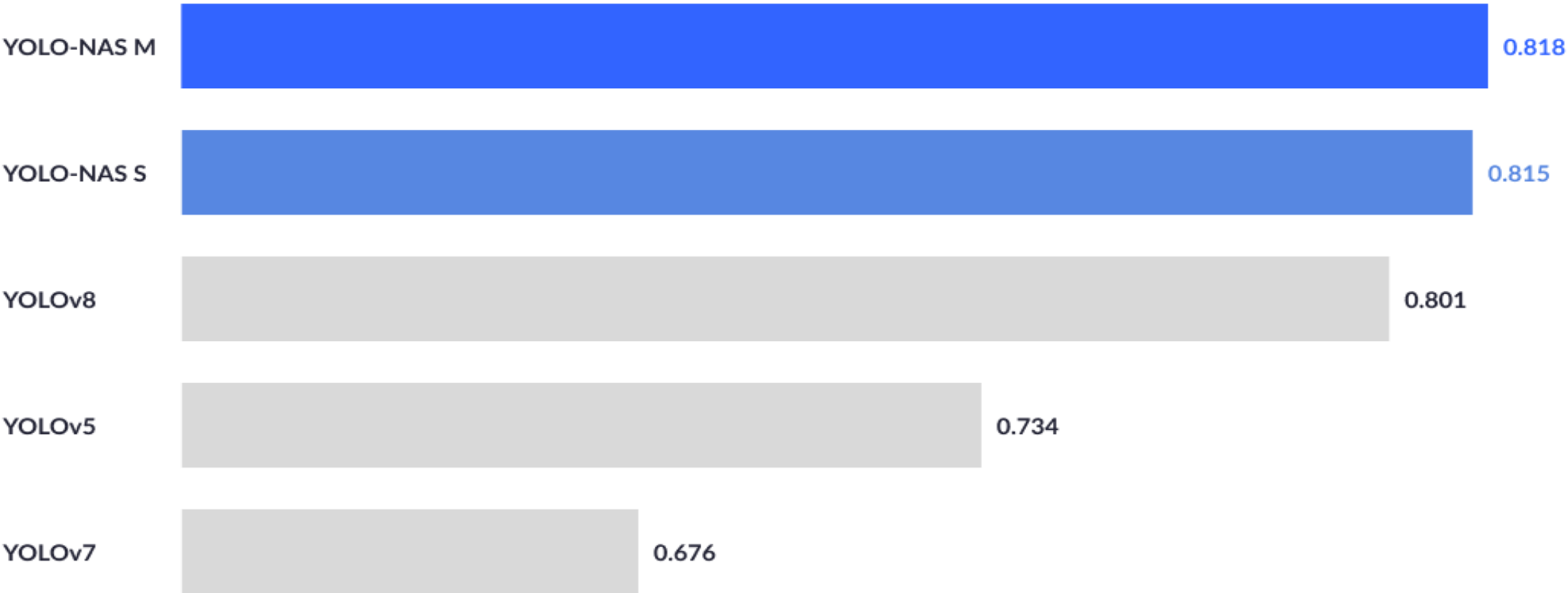


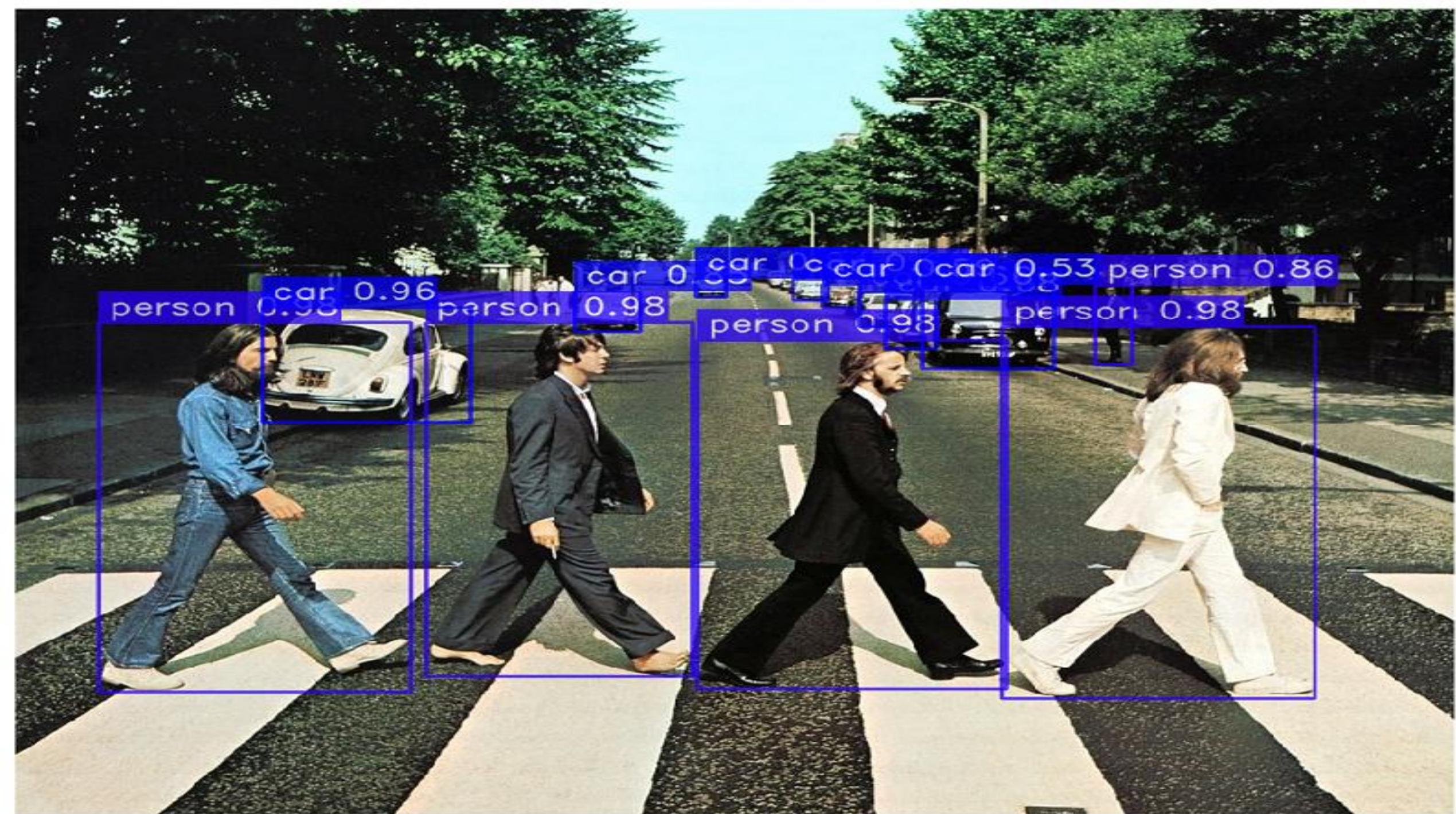
Excellent

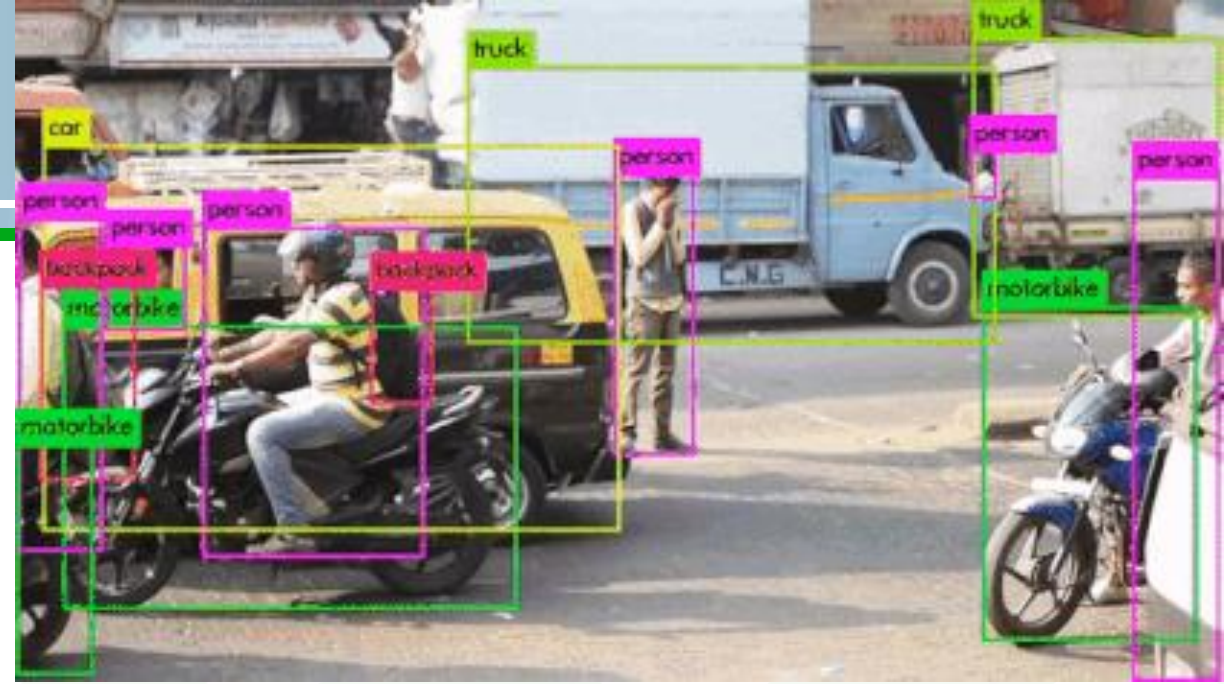
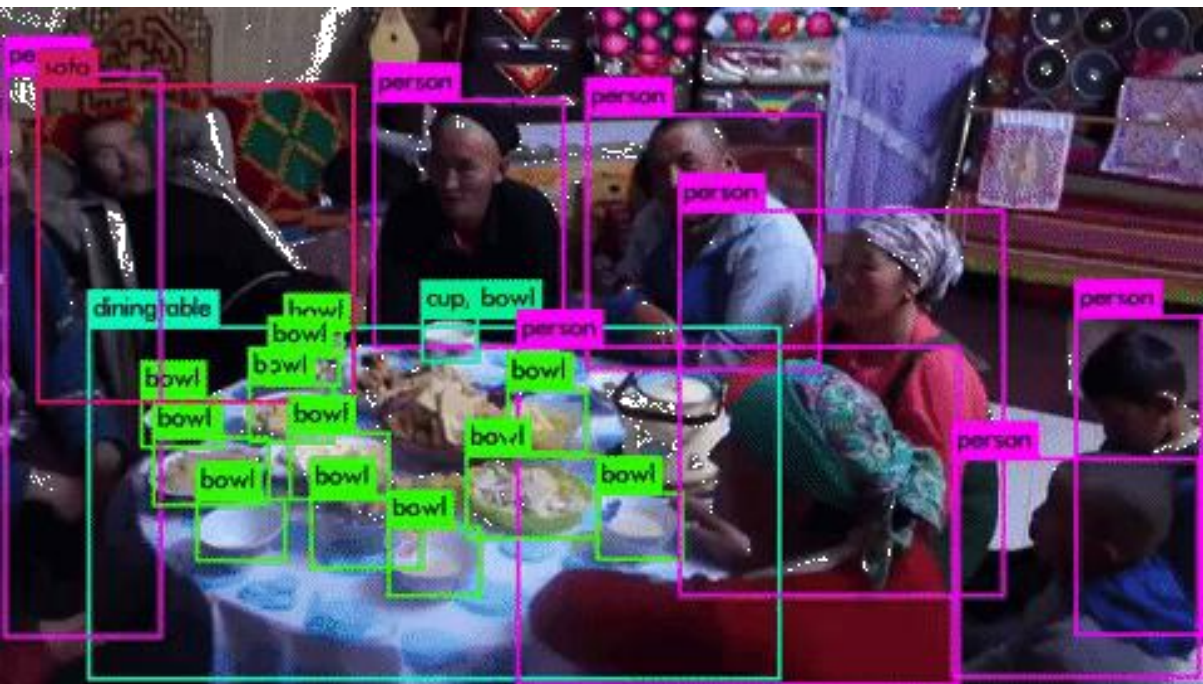
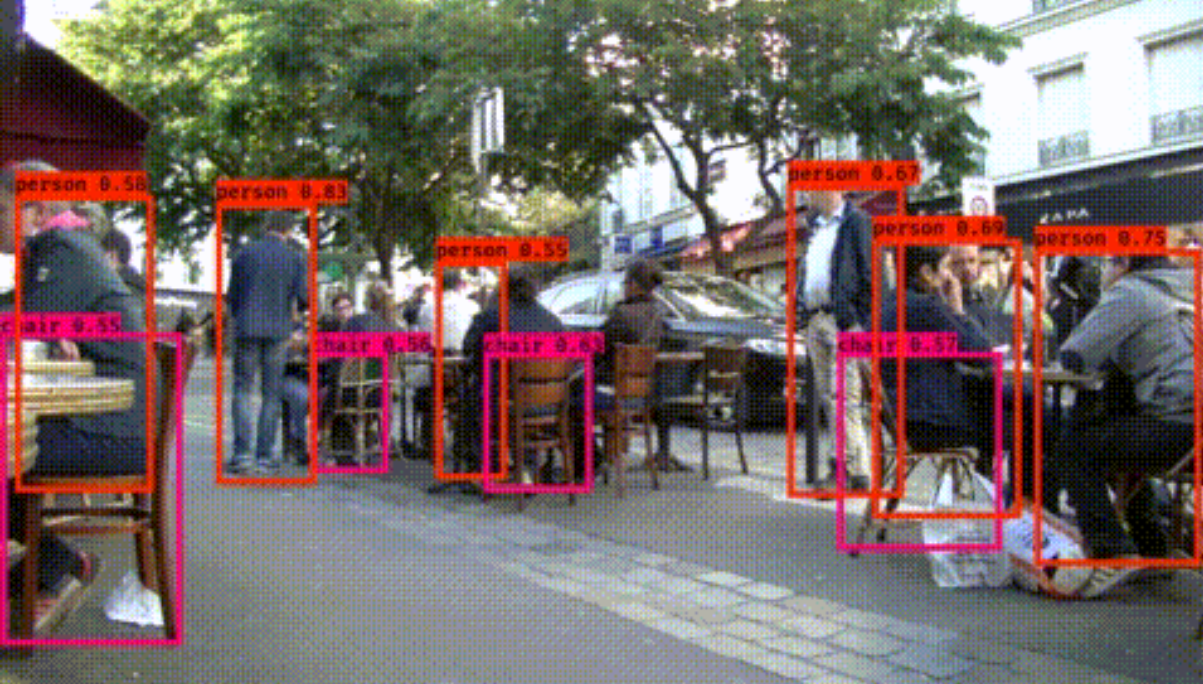
$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$



Average mAP on Roboflow-100 for YOLO-NAS vs other models







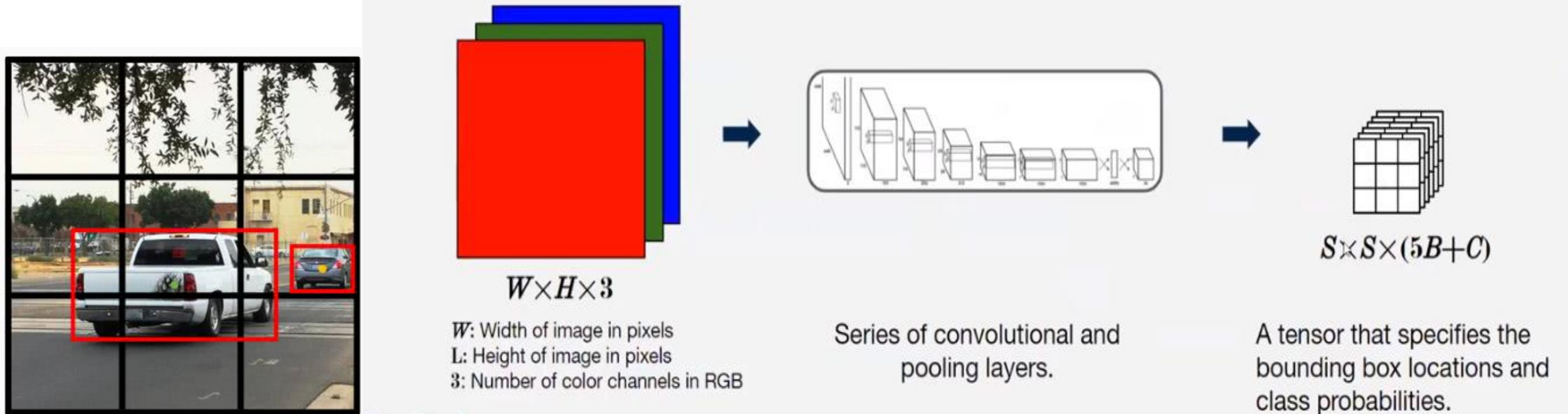
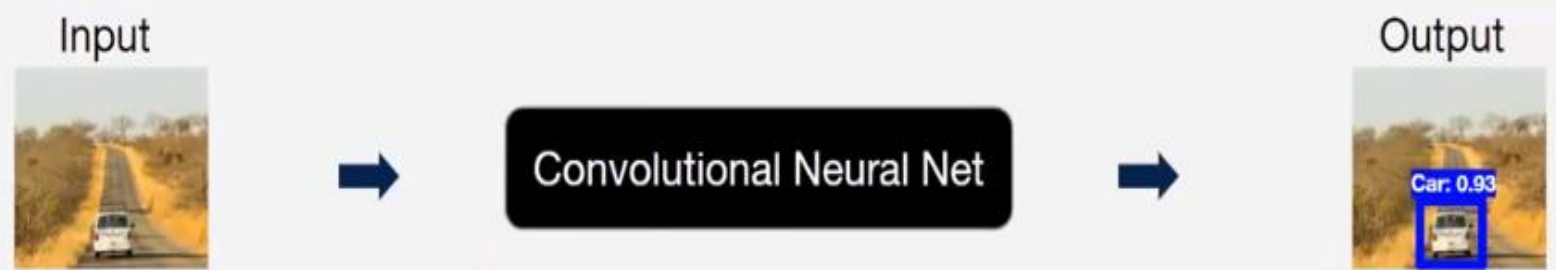
<https://storage.googleapis.com/openimages/web/visualizer/index.html?set=train&type=detection&c=%2Fm%2F09kx5>

Elephant 0.84



Elephant 0.69





- The number of **grids** in an image depends on the **grid size** and the **image size**.
- YOLO models like **YoloV3** use feature maps of different sizes, commonly 13x13, 26x26, and 52x52.
- For example, with a 416x416 pixel image, the 13x13 feature map in YoloV3 has cells that are 32x32 pixels each.
- Therefore, with a 416x416 image, the 13x13 feature map contains **169 grid cells** ($13 \times 13 = 169$), each representing a region of 32x32 pixels.



YOLO

Performance

Detection (COCO)

Segmentation (COCO)

Classification (ImageNet)

Pose (COCO)

OBB (DOTAv1)

See [Detection Docs](#) for usage examples with these models trained on [COCO](#), which include 80 pre-trained classes.

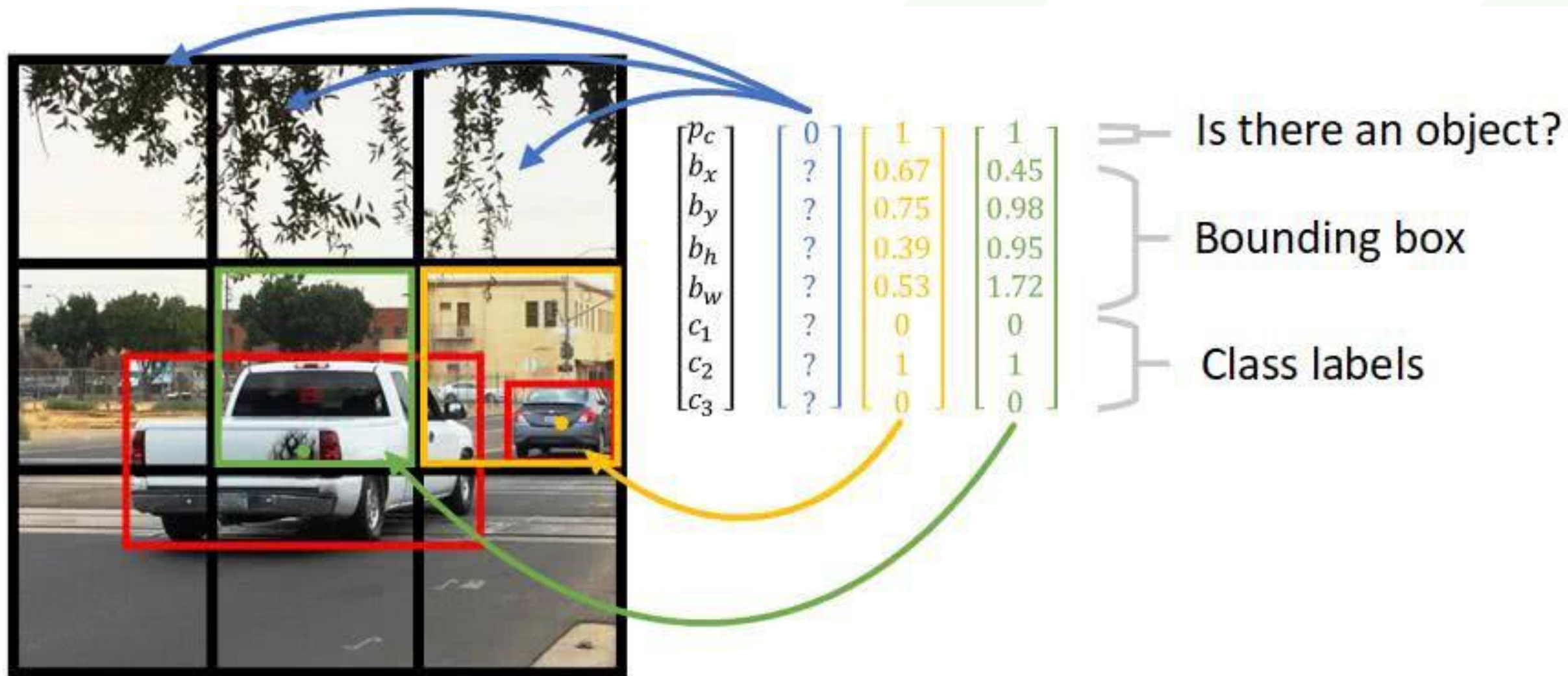
Model	size (pixels)	mAP ^{val} 50-95	Speed CPU ONNX (ms)	Speed T4 TensorRT10 (ms)	params (M)	FLOPs (B)
YOLO11n	640	39.5	56.1 ± 0.8	1.5 ± 0.0	2.6	6.5
YOLO11s	640	47.0	90.0 ± 1.2	2.5 ± 0.0	9.4	21.5
YOLO11m	640	51.5	183.2 ± 2.0	4.7 ± 0.1	20.1	68.0
YOLO11l	640	53.4	238.6 ± 1.4	6.2 ± 0.1	25.3	86.9
YOLO11x	640	54.7	462.8 ± 6.7	11.3 ± 0.2	56.9	194.9

[YOLOv11](#)

[YOLOv12](#)



YOLO

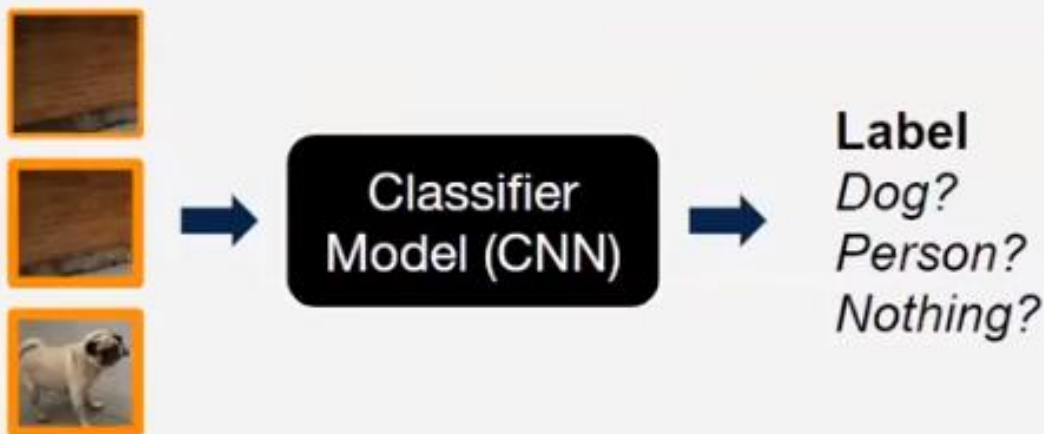


Naive Approach

1. Scan the image with a sliding window.



2. Feed the images into a classifier model to predict a label for that region.



- Slow → Not good for real time uses
- Improved version: Region-based Convolutional Neural Net (**R-CNN**)
 - Strategically selects interesting regions to run through the classifier.



YOLO

- Instead of making predictions on many regions of an image, YOLO passes the *entire* image at once into a CNN (much faster!)
- The CNN predicts the **labels, bounding boxes, and confidence probabilities** for objects in the image.



Input



Convolutional Neural Net



Output

Label

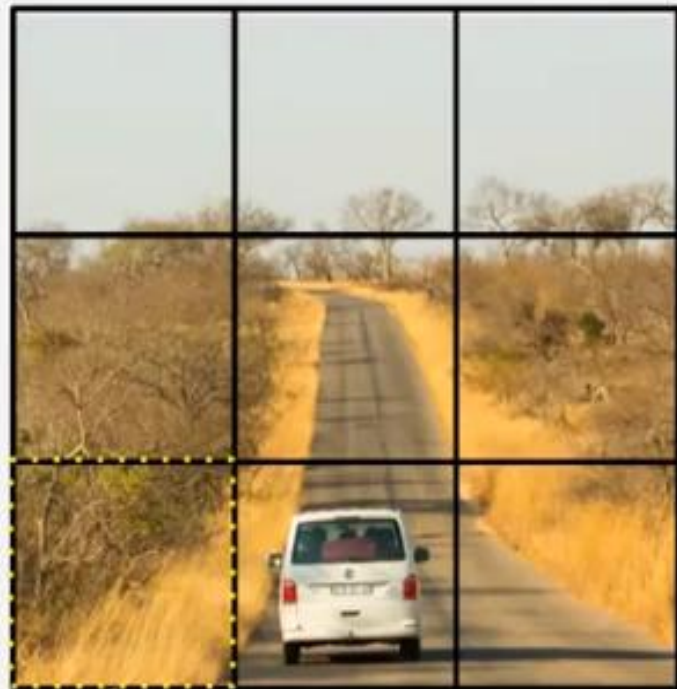
Car: 0.93

Confidence
Probability

Bounding
Box

YOLO Steps

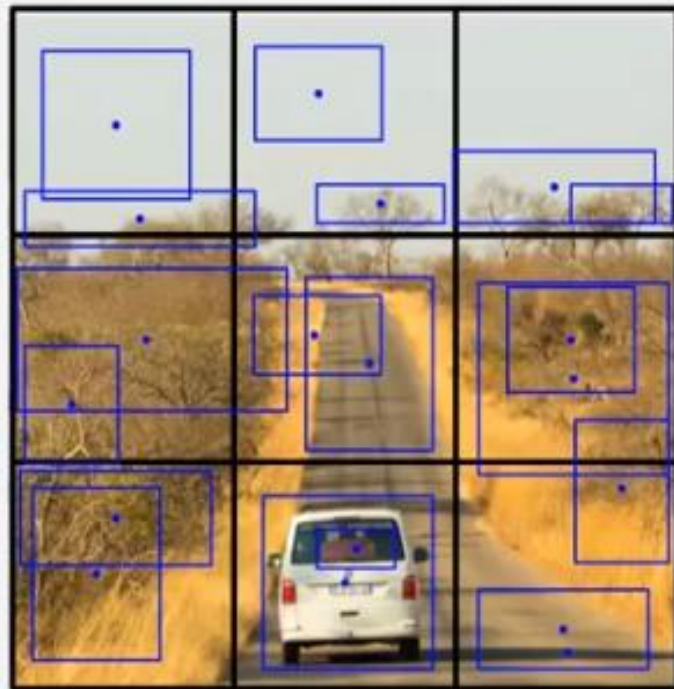
1. Divide the image into cells with an $S \times S$ grid.



$S = 3$

Cell

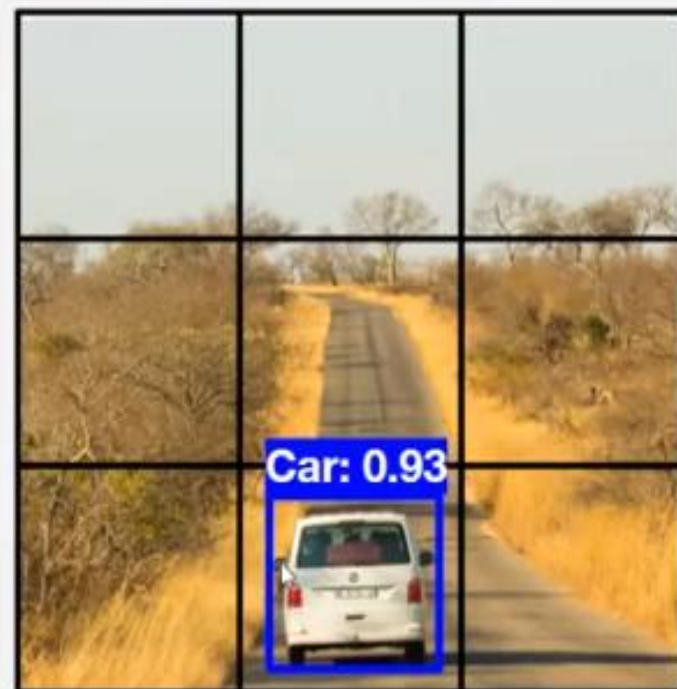
2. Each cell predicts B bounding boxes.



$B = 2$

A cell is responsible for detecting an object if the object's bounding box falls within the cell. (Notice that each cell has 2 blue dots.)

3. Return bounding boxes above confidence threshold.

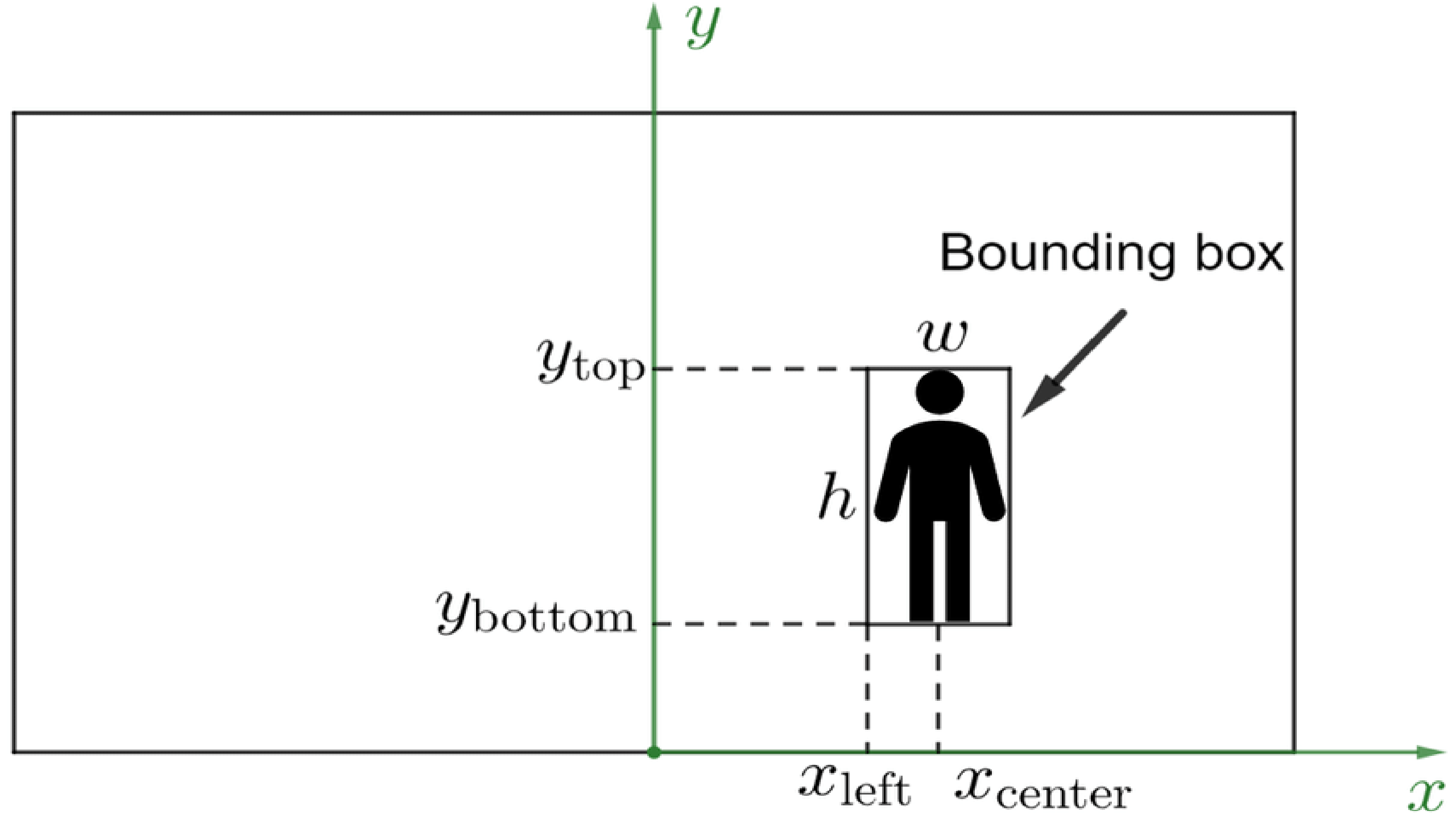




YOLO

Here's a simplified explanation of YOLO:

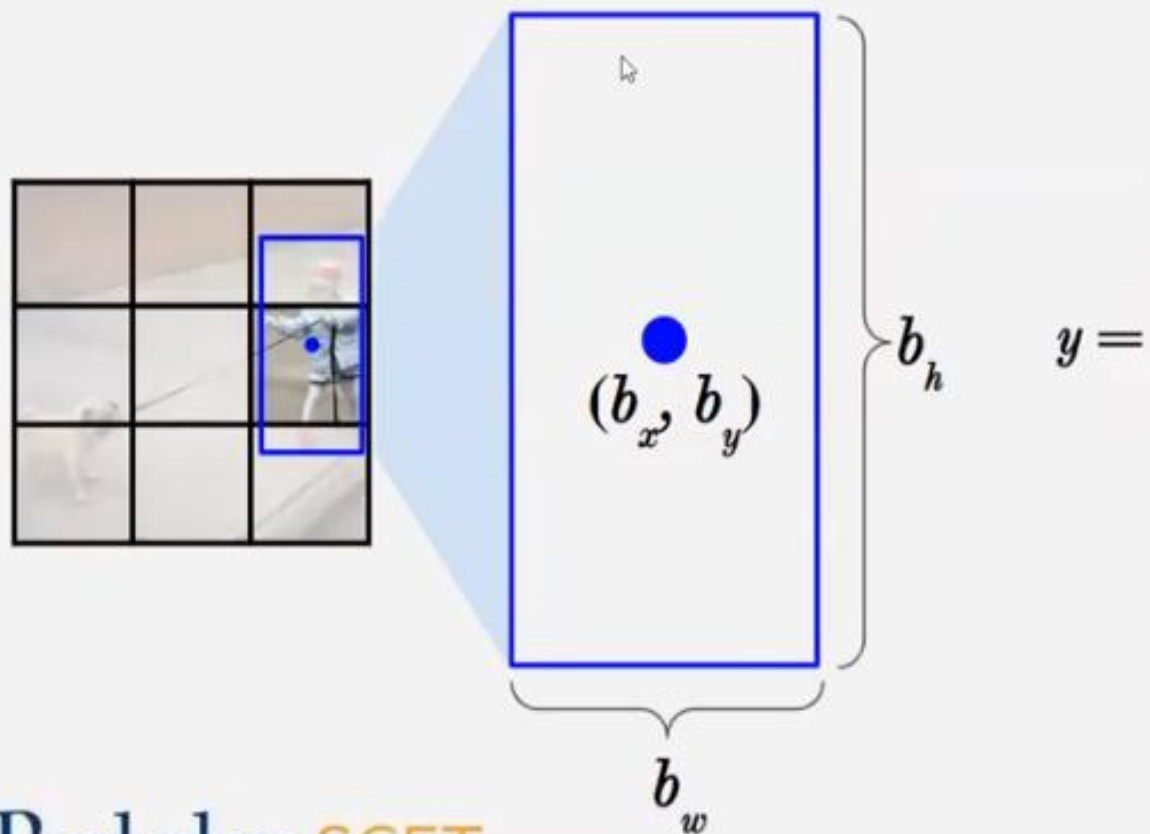
- Imagine you're playing in a garden with different objects, such as a ball, a flower, and a tree.
- Let's **divide the garden visually** into squares, like a **chessboard**.
- **Your task is to find and name the objects in the garden.** You scan each square to see if there's an object inside.
- **YOLO works similarly by dividing an image into a grid of squares.**
- Each grid cell represents a square. **YOLO** then **detects objects in each grid cell**.
- YOLO uses a logic to examine the pixels in each grid cell to determine if there's an object there.
- **If it finds an object, it draws a bounding box around it and assigns a name to the object.**
- This way, YOLO finds different objects in the image and labels them.
- Using this information, YOLO can **recognise** and **classify objects**. For example, if there's a dog and a car in a picture, YOLO finds and labels these objects with boxes and names.
- This grid-based approach helps YOLO scan images to find and classify objects, similar to how you find and name objects in a garden as a child.



How are bounding boxes encoded?

Let's use a simple example where there are 3×3 cells ($S=3$), each cell predicts 1 bounding box ($B=1$), and objects are either dog = 1 or human = 2.

For each cell, the CNN predicts a vector y :



$y =$

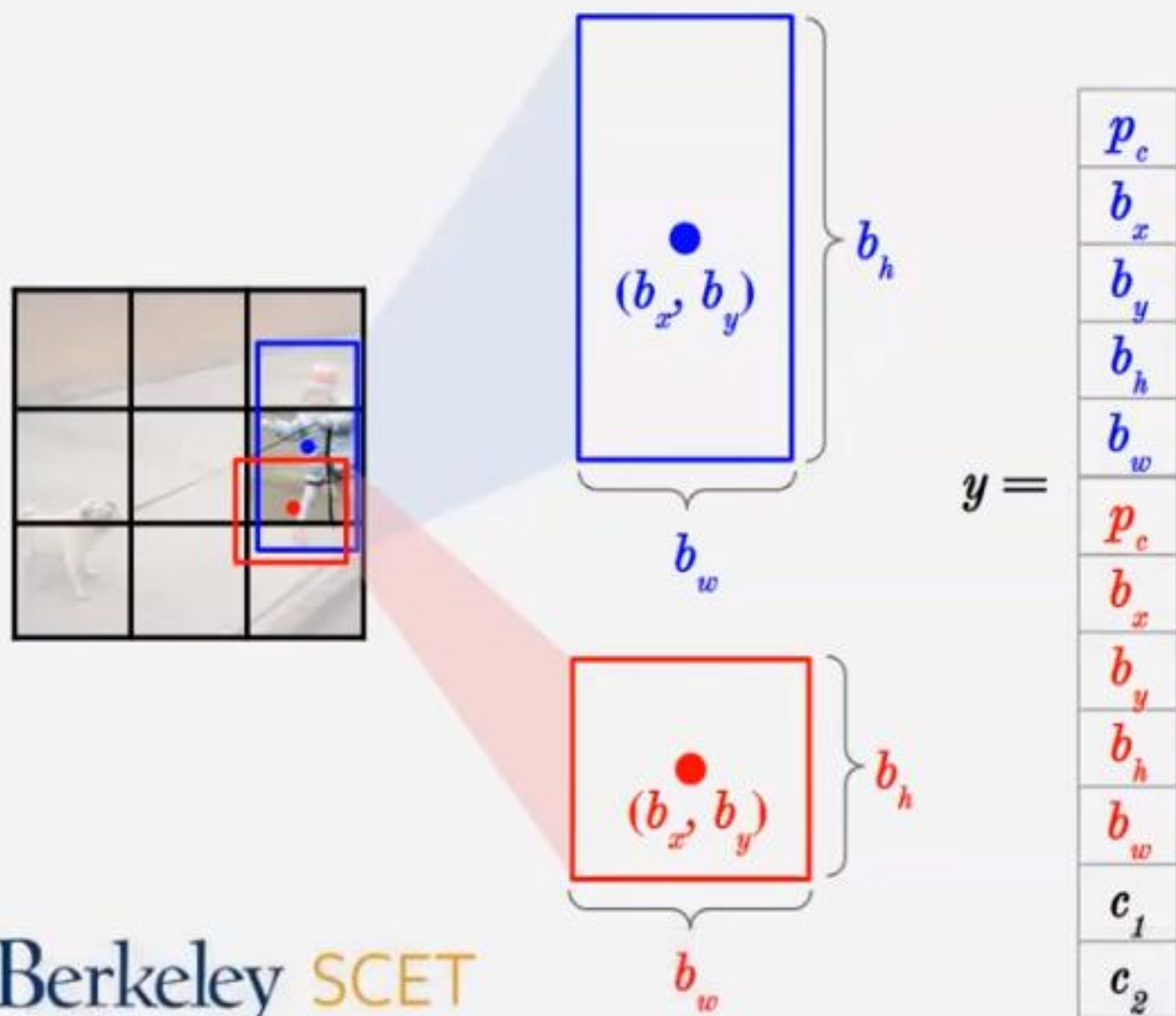
p_c	Probability the bounding box contains an object
b_x	Coordinates of the bounding box's center
b_y	
b_h	Width (height) of bounding box as a percent of the cell's width or (height)
b_w	
c_1	Probability the cell contains an object that belongs to class 1 (or 2) given the bounding box contains an object
c_2	

Example:

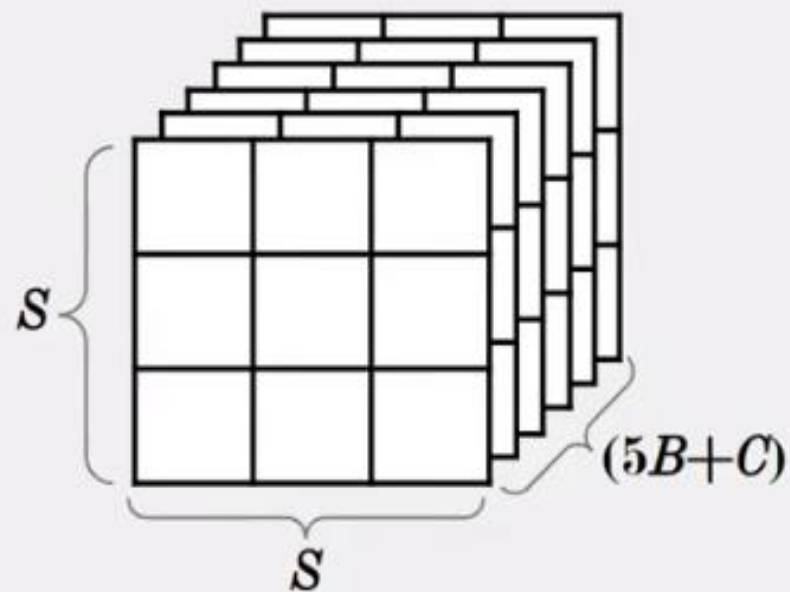
$y =$	1
	b_x
	b_y
	b_h
	b_w
	0
	1

Encoding Multiple Bounding Boxes

What happens if we predict multiple bounding boxes per cell ($B > 1$)? We simply augment y .



The CNN will predict a y for each cell, so the size of the output tensor (multidimensional "matrix") should be: $S \times S \times (5B + C)$



Notice that y has $5B + C$ elements (C is the number of classes).

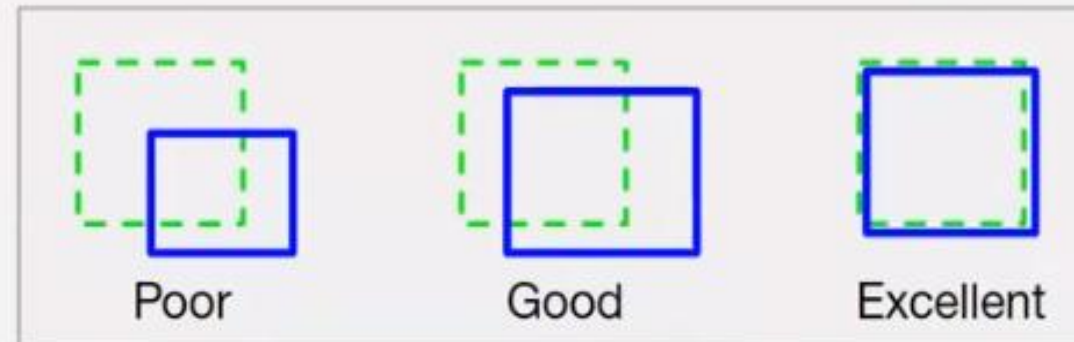
Measuring Performance with UoI

- **Union over Intersection (UoI)** measures the overlap between two bounding boxes.
- During training, we calculate the UoI between a predicted bounding box and the ground truth (the pre-labeled bounding box we aim to match)



$$\text{Union over Intersection} = \frac{\text{Area of Intersection}}{\text{Area of Union}}$$

The diagram shows two overlapping blue rectangles. The top rectangle is enclosed by a green dashed box, and the bottom rectangle is also enclosed by a green dashed box. The intersection of the two rectangles is shaded in a darker blue.



Double Counting Objects (Non-Max Suppres

- Sometimes the same object will be detected multiple times
- **Non-max suppression** solves multiple counting by removing the box with the lower confidence probability when the UoI between 2 boxes with the same label is above some threshold.



 UoI: 0.62

 UoI: 0.47

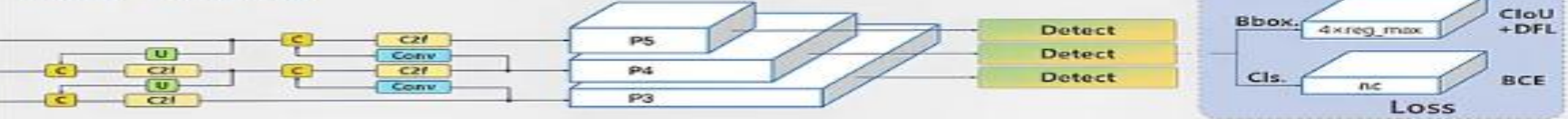


1. Identify the box with the highest confidence.
2. Calculate the UoI between the highest confidence box each of the other boxes.
3. Suppress boxes with UoI above a selected threshold (usually 0.3)

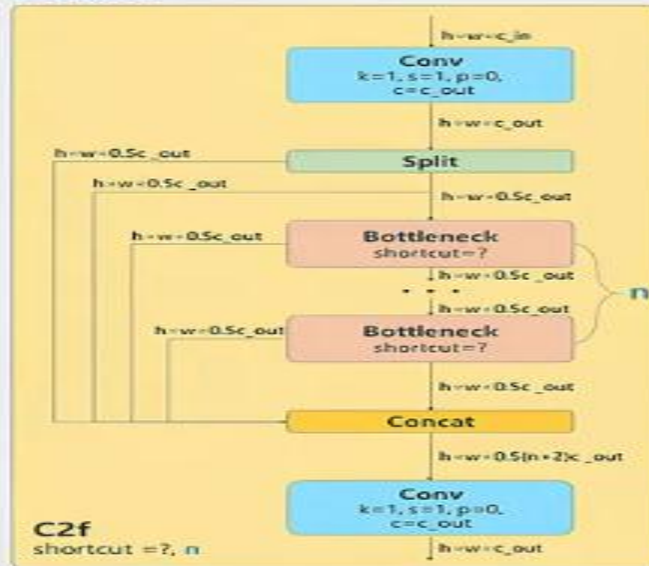
YOLOv8 Architecture



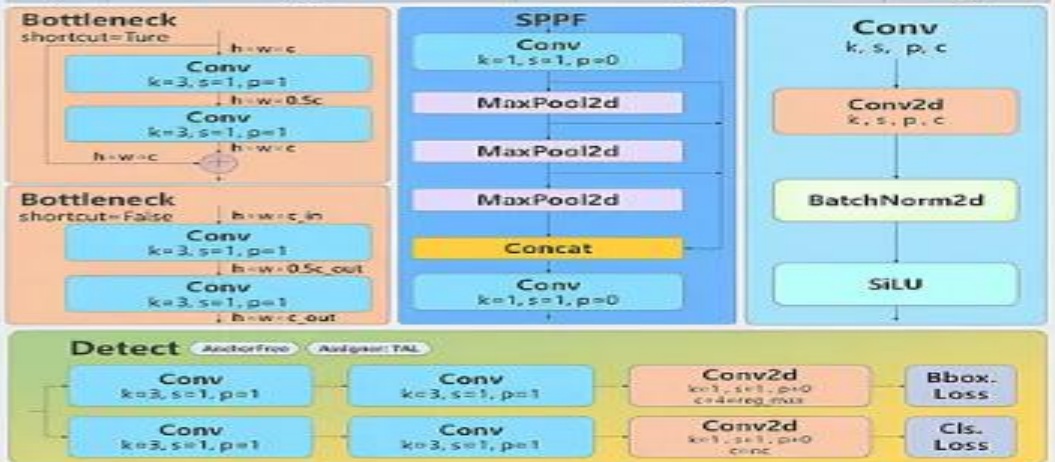
Head YOLOv8Head



Details



model	d (depth_multiple)	w (width_multiple)	r (ratio)
n	0.33	0.25	2.0
s	0.33	0.50	2.0
m	0.67	0.75	1.5
l	1.00	1.00	1.0
x	1.00	1.25	1.0



Note: height=width=channel

Backbone

<https://ruintheextinct.medium.com/yolo-models-an-objective-comparison-1566e6b6dcb7>

Head



YOLOv9 Architecture

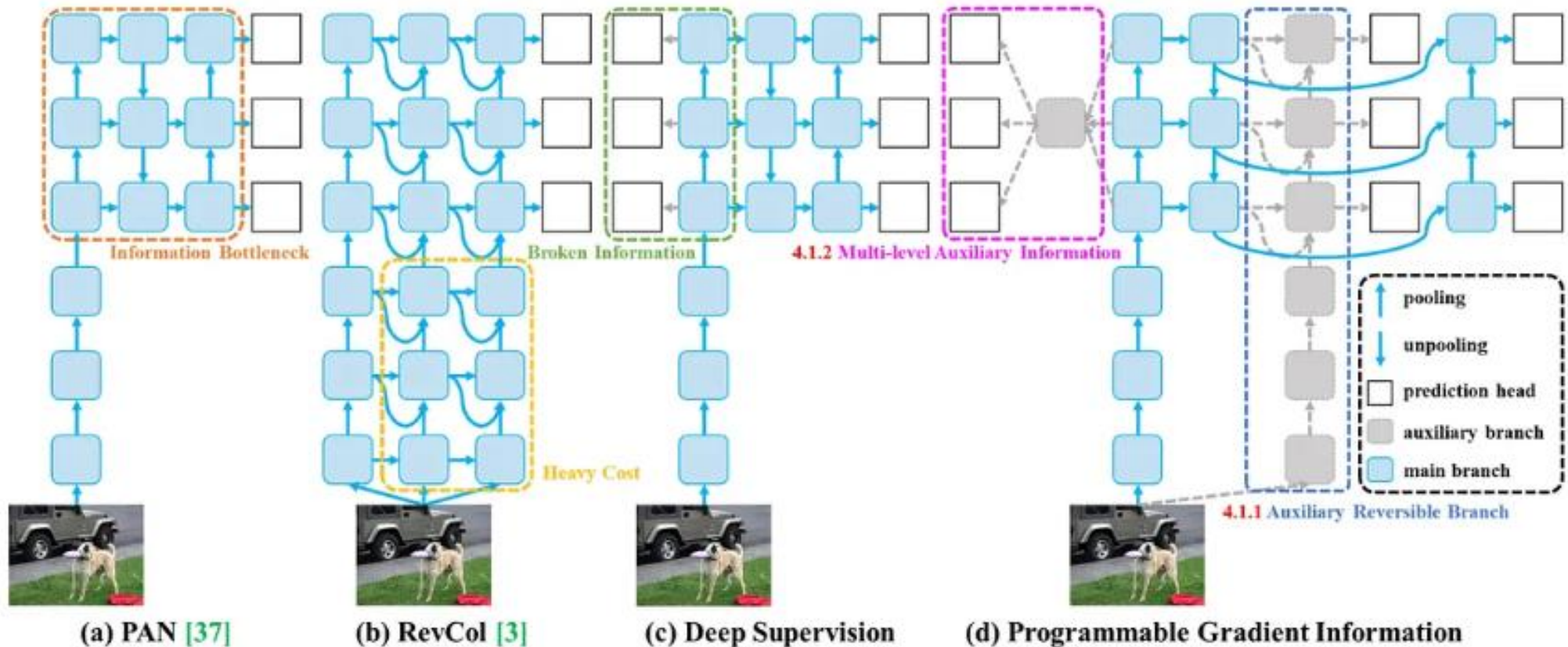
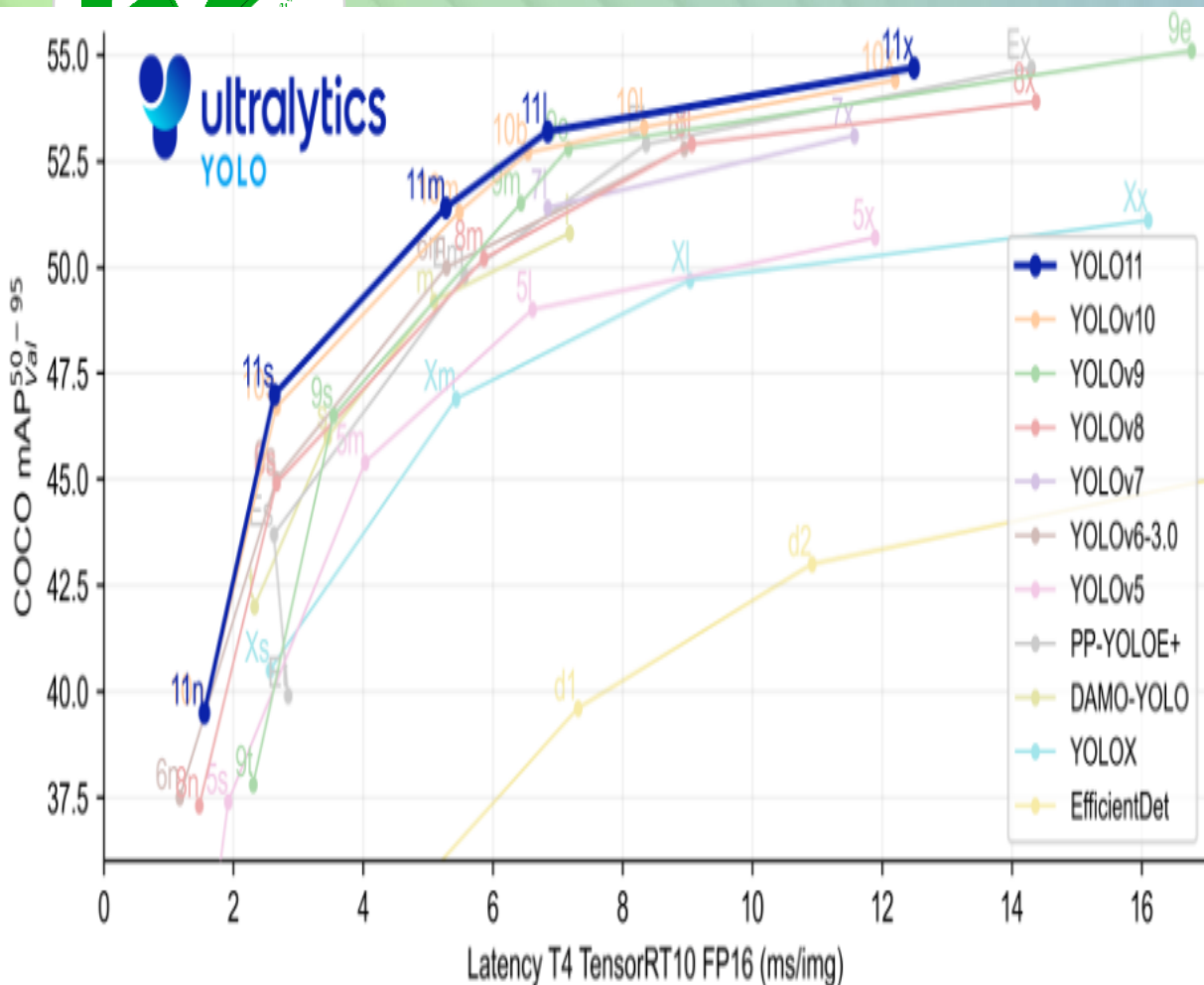


Figure 3. PGI and related network architectures and methods. (a) Path Aggregation Network (PAN)) [37], (b) Reversible Columns (RevCol) [3], (c) conventional deep supervision, and (d) our proposed Programmable Gradient Information (PGI). PGI is mainly composed of three components: (1) main branch: architecture used for inference, (2) auxiliary reversible branch: generate reliable gradients to supply main branch for backward transmission, and (3) multi-level auxiliary information: control main branch learning plannable multi-level of semantic information.



YOLO 11



Key Features

- **Enhanced Feature Extraction:** YOLO11 employs an improved backbone and neck architecture, which enhances **feature extraction** capabilities for more precise object detection and complex task performance.
- **Optimized for Efficiency and Speed:** YOLO11 introduces refined architectural designs and optimized training pipelines, delivering faster processing speeds and maintaining an optimal balance between accuracy and performance.
- **Greater Accuracy with Fewer Parameters:** With advancements in model design, YOLO11m achieves a higher **mean Average Precision** (mAP) on the COCO dataset while using 22% fewer parameters than YOLOv8m, making it computationally efficient without compromising accuracy.
- **Adaptability Across Environments:** YOLO11 can be seamlessly deployed across various environments, including edge devices, cloud platforms, and systems supporting NVIDIA GPUs, ensuring maximum flexibility.
- **Broad Range of Supported Tasks:** Whether it's object detection, instance segmentation, image classification, pose estimation, or oriented object detection (OBB), YOLO11 is designed to cater to a diverse set of computer vision challenges.

<https://www.kaggle.com/code/duygujones/sign-language-detection-using-yolo11>

<https://docs.ultralytics.com/models/yolo11/#overview>

<https://www.youtube.com/watch?v=-JXwa-WlU8&t=19s>



YOLO trained on the COCO Dataset



YOLO

[https://coco
dataset.org/
#home](https://coco dataset.org/#home)



Pretrained Models

- Images with bounding boxes pre-labeled are often used to train object detection models
- [COCO \(Common Objects in Context\)](#) - a popular computer vision database of images with 80 labeled objects



YOLO Implementation
(CNN)

Pretrained Model
with COCO



Pineapples and cantaloupes are not in COCO so they are not recognized.

COCO Pretrained Labels

Example labels: person, car, motorbike, traffic light, dog, cat, sheep, spoon, cup, sandwich, keyboard, chair, toaster, toothbrush, sports ball...

Dataset examples



Image from <https://cocodataset.org/#home>

COCO Pretrained Labels

person	fire hydrant	elephant	skis	wine glass	broccoli	diningtable	toaster
bicycle	stop sign	bear	snowboard	cup	carrot	toilet	sink
car	parking meter	zebra	sports ball	fork	hot dog	tvmonitor	refrigerator
motorbike	bench	giraffe	kite	knife	pizza	laptop	book
aeroplane	bird	backpack	baseball bat	spoon	donut	mouse	clock
bus	cat	umbrella	baseball glove	bowl	cake	remote	vase
train	dog	handbag	skateboard	banana	chair	keyboard	scissors
truck	horse	tie	surfboard	apple	sofa	cell phone	teddy bear
boat	sheep	suitcase	tennis racket	sandwich	pottedplant	microwave	hair drier
traffic light	cow	frisbee	bottle	orange	bed	oven	toothbrush

Custom Models

Are the objects to detect in COCO?

Yes

Pretrained Model

1. Download model. Some pretrained YOLO models:
 - [ImageAI](#) (easy-to-use, lightweight YOLO implementation)
 - [Darknet](#) (trained by the author of YOLO)

No

Train a Custom Model

1. Finding images of the objects.
2. Label bounding boxes.
3. Train your YOLO model:
 - i. Implement your own model using OpenCV, Tensorflow/Keras
 - ii. Use a library (such as [ImageAI](#)'s custom training methods)



What will we do?

1. We will be labelling our images by using **MakeSense** (or Labellmg)
2. We apply image generation on **Roboflow** by using our labelled images.
3. We will move our data to **YOLO** models (on **Colab**), integrate our data into YOLO, train (use) that pre-trained YOLO model and get results on images.
4. We will give a **video** to our trained YOLO and get results for the video.